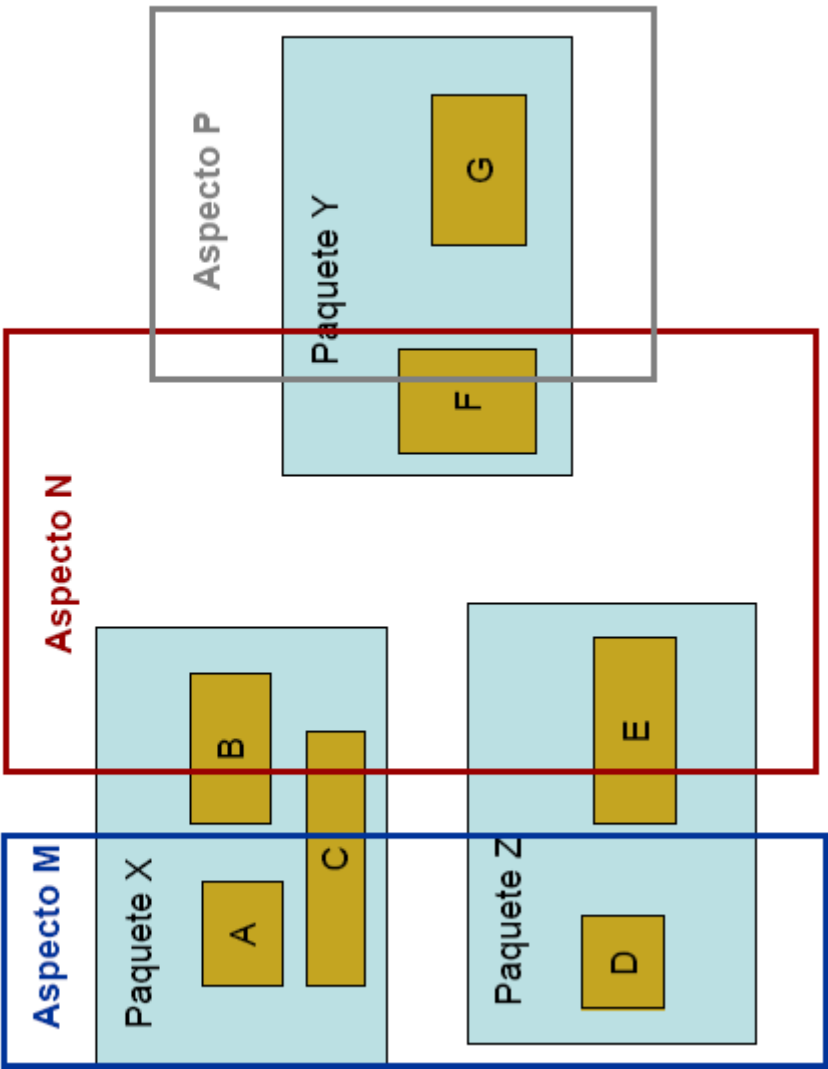


[illegible]

- Son muchos los intereses que un equipo de desarrollo de software debe fijar su atención: entender el problema, entender su entorno, gestión del proyecto, equipo de desarrollo, aspectos técnicos, entre otros. Los intereses referidos a aspectos técnicos como seguridad, persistencia, presentación, manejo de errores, etc. Han dado origen al paradigma orientado a aspectos.

- El enfoque orientado a aspectos define un mecanismo que ayuda a resolver problemas de codificación en los requisitos, los cuales son un código disperso (*scattered*) y diseminado (*tangled*), que no se resuelven fácilmente usando el enfoque orientado a objetos



La evolución de la ingeniería de software

1ª Generación: Código spaghetti. *Esta generación se caracterizó por que no* existía una clara separación entre datos y funcionalidad. Como consecuencias de esta falta de separación se tiene un código resultante es complejo y una alta dificultad para realizar cambios

2ª y 3ª Generación: Descomposición funcional: *En estas generaciones se hace* visible una clara división entre los datos y la funcionalidad, identificando las partes más manejables como funciones que se definen en el dominio del problema. La facilidad de integración de nuevas funciones, el hecho que las funciones queden algunas veces poco claras debido a la utilización de datos compartidos y que los datos quedan esparcidos por todo el código, son algunas de las características que se observan en esta generación.

4ª Generación: Orientación a objetos: En esta generación aparece el modelo de objetos, el cual utiliza el principio de descomposición funcional, ajustándose mejor al dominio del problema. Algunas de las características que se observan en esta generación son:

- Provee una fácil la integración de nuevos datos.
- Las funciones quedan esparcidas por todo el código.
- Con frecuencia, para realizar la integración de nuevas funciones hay que modificar varios objetos.
- Enmarañamiento de los objetos en funciones de alto nivel que involucran a varias clases.

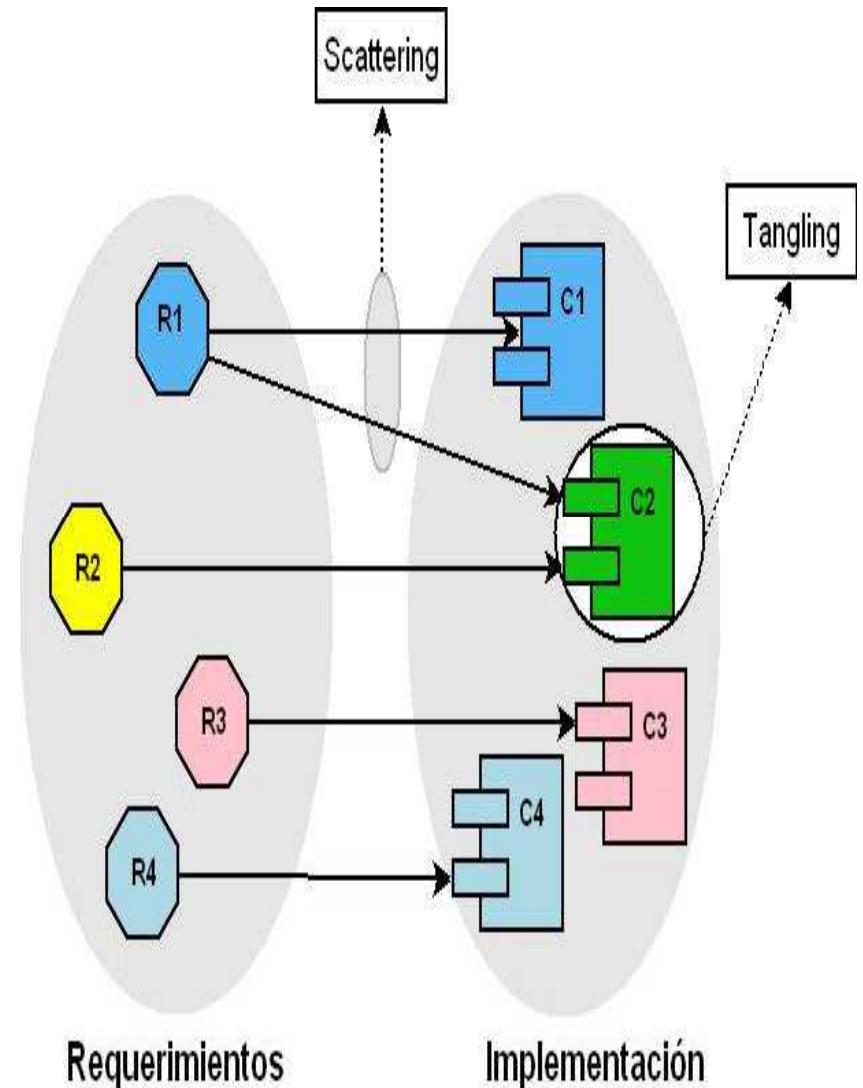
5ª Generación: ¿Orientación a aspectos?: Separa los componentes funcionales y los atributos de calidad unos de otros, proporcionando mecanismos que hagan posible abstraerlos y componerlos para formar todo el sistema.

Antecedentes del Desarrollo Basado en Aspectos

- El PO está basado sobre los conceptos de herencia, encapsulamiento y polimorfismo.
- El tangling y su consecuente alto acoplamiento entre los componentes funcionales de un sistema acaban con la modularidad del código, limitando su reutilización y hace los programas más propensos a errores. El tangling acaba con la calidad del software, y los beneficios asociados con la PO no pueden ser utilizados completamente.
- Los casos de uso han sido universalmente adoptados para la especificación de requisitos. Los casos de uso comienzan en requisitos, son traducidos a colaboraciones en análisis y diseño, y a casos de prueba. La codificación de un componente o una clase requiere de nosotros para mezclar el código derivado de varios casos de uso, los mecanismos de extensión soportados cuando se trabaja con casos de uso, con el UML actual no es soportado entre los elementos de análisis y diseño tampoco en colaboraciones, componentes, clases ni en la implementación en ambientes como JAVA o C#.
- El principal problema es la limitación de los lenguajes usados actualmente. La programación orientada a aspectos es el enlace faltante, esto permitirá cortar un sistema claramente, caso de uso por caso de uso sobre muchos modelos y los casos de uso permanecen separados, logrando así, la separación de intereses

El problema: tangling y scattering

- Estos fenómenos se presentan por la incapacidad del paradigma orientado a objetos para soportar la separación de intereses, no sólo en las etapas finales, sino desde las etapas tempranas del ciclo de vida del software.
- *Scattering* como la necesidad de diseminar un requisito a través de muchos componentes del sistema.
- El *tangling*, por el contrario, consiste en la necesidad de hacer que un sólo componente del sistema tenga que realizar todo un conjunto de requisitos.
- Dificultan la reutilización, aumentan el acoplamiento entre componentes al tiempo que aumentan la complejidad del sistema y hacen que el software pierda capacidad para evolucionar.
- Si bien se ha identificado al *tangling* y al *scattering* como principales fallas del paradigma orientado a objetos, se hace necesario identificar cuándo aparecen estos problemas.



La base del paradigma: Clases y Aspectos

- Un sistema orientado a aspectos puede verse como una combinación de la *funcionalidad básica* y el *comportamiento aspectual*, los cuales pueden ser combinados por medio de *puntos de enlace*.

Funcionalidad básica

- El paradigma orientado a aspectos utiliza toda la potencia de la orientación a objetos para soportar la funcionalidad base del sistema. (ej.: gestión de 30 inventarios, el pago de una nomina, entre otros).
- La funcionalidad básica está determinada por el dominio del problema.

Comportamiento aspectual

- El enfoque original con el cual surgió la orientación a aspectos identificaba el comportamiento aspectual sólo con conceptos técnicos tales como la persistencia, la gestión de errores, la sincronización o la comunicación de procesos. Hoy, el comportamiento aspectual es llevado más allá de las características técnicas del sistema y empiezan a identificarse conceptos del dominio del problema que tiene comportamiento aspectual y que *cruzan el sistema*.
- Los lenguajes orientados a aspectos definen una nueva unidad de programación de software para encapsular las funcionalidades que cruzan todo el código.

Los lenguajes orientados a aspectos pueden ser de propósito general o de dominio específico:

- **Lenguajes de dominio específico:** Soportan uno o más de los aspectos considerados (distribución, manejo de errores, etc.), pero no pueden soportar otros aspectos distintos de aquellos para los que fueron diseñados. Los lenguajes de aspectos de dominio específico normalmente tienen un nivel de abstracción mayor que el lenguaje base y, por tanto, expresan los conceptos del dominio específico del aspecto en un nivel de representación más alto. (COOL (COOrdination Language), RIDL(Remote Interaction and Data transfers Language).
- **Lenguajes de propósito general:** se diseñaron para ser utilizados con **cualquier** clase de aspecto, no solamente con aspectos específicos. Por lo tanto, no pueden imponer restricciones en el lenguaje base. Principalmente soportan la definición separada de los aspectos proporcionando unidades de aspectos. Normalmente tienen el mismo nivel de abstracción que el lenguaje base y también el mismo conjunto de instrucciones, ya que debería ser posible expresar cualquier código en las unidades de aspectos. (AspectJ, que utiliza Java como base, y las instrucciones de los aspectos también se escriben en Java).

Puntos de enlace

- Para poder combinar la *funcionalidad base con el comportamiento aspectual* se requiere determinar algunos puntos comunes, que son los que se conocen como *puntos de enlace (join points)*.
- Los puntos de enlace son una clase especial de interfaz entre los aspectos y los módulos del lenguaje de componentes. Son los lugares del código en los que éste se puede aumentar con comportamientos adicionales. Estos comportamientos se especifican en los aspectos.

Tejedor (weaver)

- La interacción entre clases y aspectos se hace posible a través de un tercer componente, el cual conocemos como *tejedor*. *El tejedor es el encargado de realizar la mezcla de la funcionalidad base con el comportamiento aspectual.*
- *Las* clases y los aspectos se pueden mezclar de dos formas distintas: de manera estática y de manera dinámica.
- **Tejido estático.** Implica modificar el código fuente de una clase insertando sentencias en estos puntos de enlace. Es decir, que el código del aspecto se introduce en el de la clase. Se evita un impacto negativo en el rendimiento de las aplicaciones, pero se hace difícil identificar los aspectos en el código una vez ya se ha tejido.
- **Tejido dinámico.** El entrelazado dinámico requiere que los aspectos existan y estén presentes de forma explícita tanto en tiempo de compilación como en tiempo de ejecución. A partir de una interfaz de reflexión, el tejedor es capaz de añadir, adaptar y borrar aspectos de forma dinámica, si así se desea, durante la ejecución.

Aplicaciones bajo el paradigma orientado a aspectos

- Para construir un sistema orientado a aspectos tenemos tres elementos principales:
 - Clases, las cuales modelan la funcionalidad base
 - Aspectos
 - Tejedor
- En el esquema tradicional, la construcción de la aplicación se realiza a través de un compilador o intérprete, el cual simplemente toma el código de la funcionalidad base y lo traduce a un lenguaje entendible por la maquina.

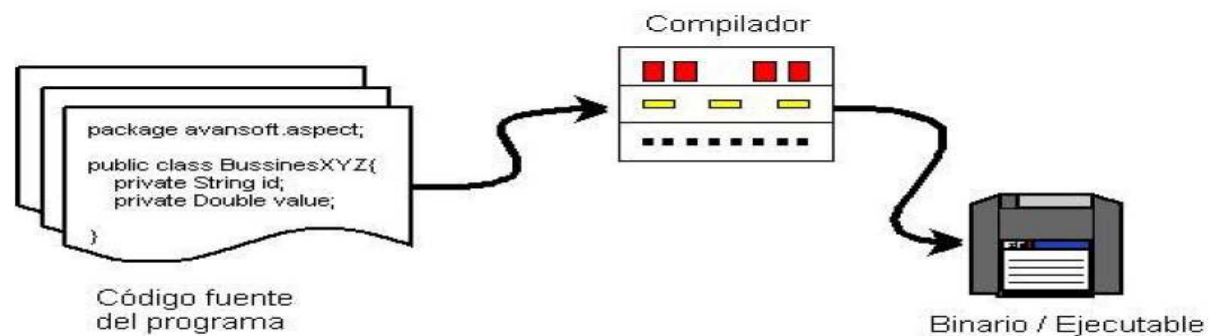


Figura 2. Construcción de aplicaciones bajo el esquema tradicional. [20]

- La construcción de una aplicación bajo el enfoque orientado a aspectos requiere la combinación del código de la funcionalidad base con los distintos módulos que implementan los aspectos. Esta combinación es realizada por el *tejedor*. Cada aspecto codificado con un lenguaje distinto.

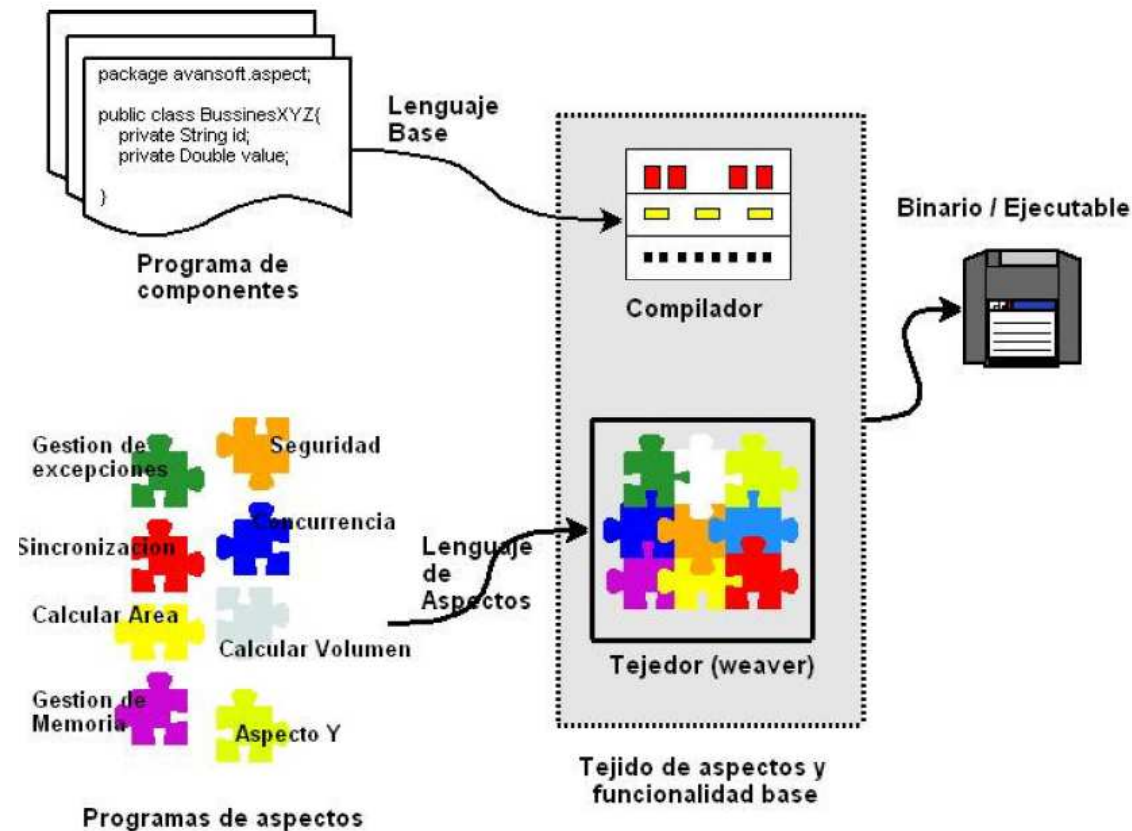
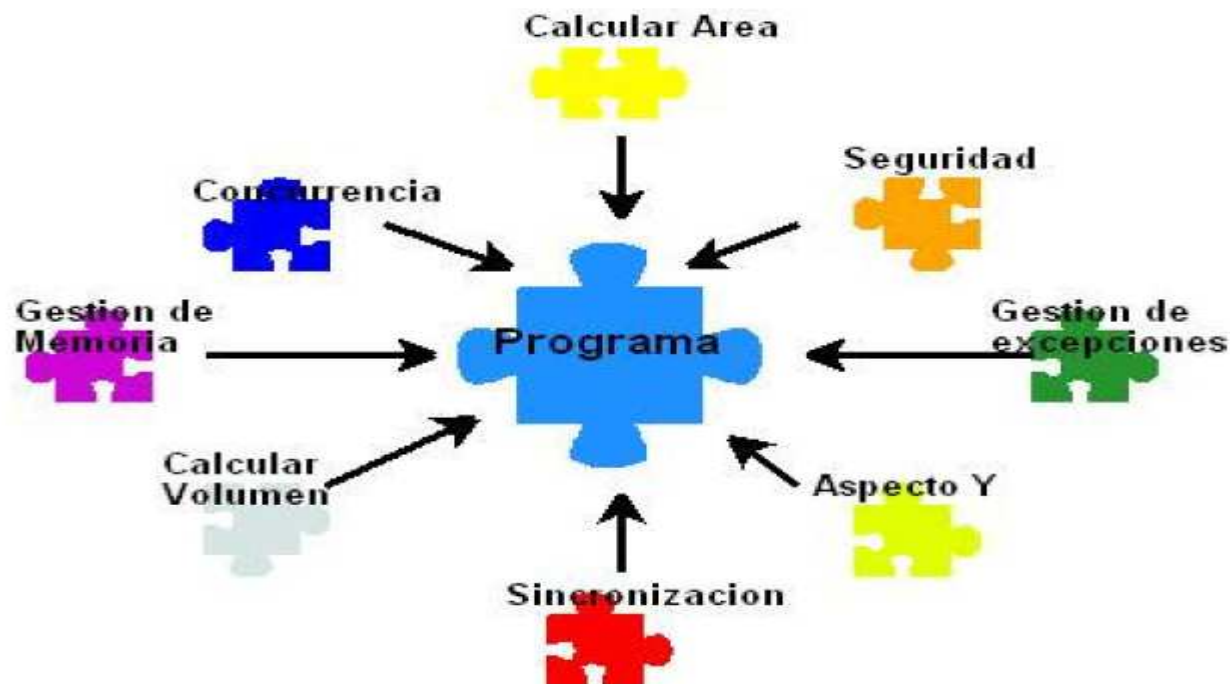


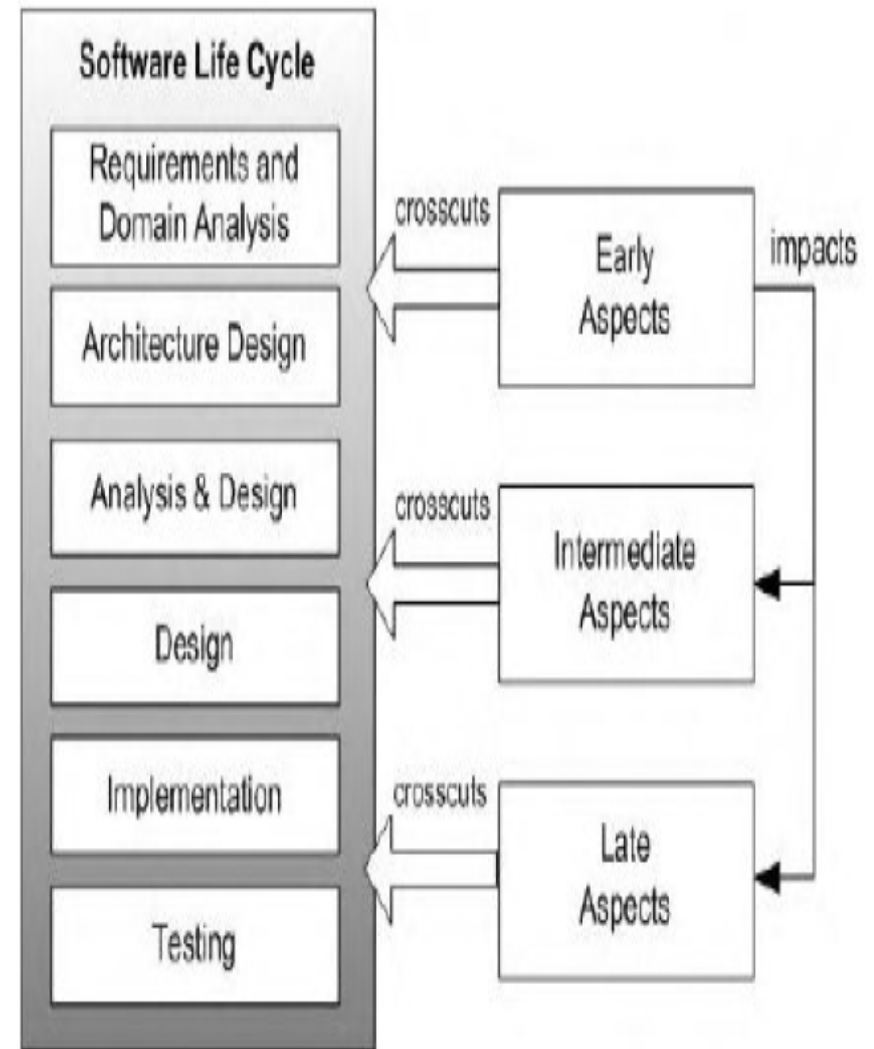
Figura 3. Construcción de aplicaciones bajo el paradigma orientado a aspectos. [20]

- El paradigma aspectual nos permite ver el sistema como un conjunto conformado por un modelo de objetos y varios modelos de aspectos.
- El modelo de objetos es el encargado de *implementar la funcionalidad base del sistema*, mientras que el modelo de aspectos nos permite implementar las características no funcionales (distribución, manejo de errores, sincronización entre otras) y nos permite implementar *aspectos del dominio del problema* (Ej.: reglas, normas, registraciones, etc.)



Aspectos en etapas tempranas

- Todos los sistemas, sin importar el paradigma bajo el cual se encuentren necesitan estar bien diseñados, aplicando *buenas prácticas de ingeniería de software*. El análisis y el diseño de un sistema son tan importantes como la misma implementación, tanto así que estas fases son consideradas como las que más contribuyen al éxito de un proyecto.
- Considerar las etapas tempranas del ciclo de vida del software permite tener clara la estructura final del sistema, de igual forma que reduce el esfuerzo de los programadores ya que desde el principio se tienen definidos los caminos y procesos que se deben seguir.
- La necesidad de brindar un adecuado soporte a través de todo el ciclo de vida del software es necesario establecer una relación entre aspectos tempranos, intermedios y finales, dando lugar a una propuesta de desarrollo en la que los aspectos son elementos relevantes a lo largo del proceso de desarrollo y que se conoce como AOSD (Aspect Oriented Software Development).



Definiciones básicas en AOSD

- **Concern (Interés).** Son intereses que pertenecen al desarrollo del sistema, operaciones o cualquier otro aspecto crítico o importante para uno o más de los stakeholders.
- Los concerns incluyen consideraciones del sistema tales como desempeño, disponibilidad, seguridad, distribución y capacidad de evolución.
- Algunos concerns son localizados en un lugar particular en un sistema emergente, algunos se refieren a una propiedad medible de un sistema como un todo, otros son estéticos y otros son comportamientos sistemáticos.

Definiciones básicas en AOSD

- **Separation of concerns (Separación de interese).** *Hace referencia a la comprensión de los conceptos del sistema como unidades de software independientes.* Este principio trae asociados múltiples beneficios para el desarrollo de software, incluyendo un mejor análisis, mejor entendimiento del sistema, mayor adaptabilidad, un alto grado de reusabilidad y mantenibilidad.
- El objetivo de la separación de intereses es dar representaciones independientes a intereses independientes. La efectiva separación de intereses obtiene ventajas a través del ciclo de vida del software, pero es especialmente importante extender la vida útil de un sistema con respecto a evolución y otros pequeños cambios.
- Los intereses que tienen representaciones independientes pueden ser cambiados de manera independiente, simplificando los cambios y minimizando los costos, esfuerzo, e impacto.
- Para el proceso software la separación de intereses puede dar flexibilidad en términos de asignación y programación de tareas. Adicionalmente, puede proveer una base para gestión de la configuración y la familia de productos de gestión.
Separación de intereses es una técnica clave para la ingeniería de software para la creación de componentes de software evolutivos.

Crosscutting concern (Interés de corte transversal)

- Los intereses de corte transversal son aquellas propiedades o conceptos de un sistema que tienden a estar presentes en varios componentes funcionales (distribución y sincronización), sin embargo existen concerns de naturaleza funcional que también son de corte transversal.
- Los intereses de corte transversal, son, muchas veces, como aspectos críticos del problema de composición de software.
- Se destacan los intereses de corte transversal como la principal causa de muchas complicaciones en la arquitectura de software (carencia de abstracción para la separación y combinación de intereses de varios tipos en especificaciones de arquitectura).
- Implicaciones de los intereses de corte transversal al ser tratados desde los paradigmas no aspectuales:
 - Bajos niveles de trazabilidad entre los artefactos de la fase de requisitos y la fase de implementación.
 - Baja productividad en el proceso de desarrollo de software.
 - Bajo grado de reutilización de código fuente.
 - Bajos niveles de adaptabilidad y evolución de los sistemas.
 - Baja calidad en el software.

Aspect (Aspecto)

- “Un aspecto es una unidad que se define en términos de información parcial de otras unidades” (Demeter en 1995).
- “Un aspecto es una unidad modular que se dispersa por la estructura de otras unidades funcionales. Los aspectos existen tanto en la etapa de diseño como en la de implementación. Un aspecto de diseño es una unidad modular del diseño que se entremezcla en la estructura de otras partes del diseño.
- Un aspecto de programa o de código es una unidad modular del programa que aparece en otras unidades modulares del programa”. (Gregor Kiczales 2000).
- Un aspecto es una unidad modular diseñada para implementar un concern. La definición de un aspecto puede contener algún código y las instrucciones dónde cuándo y cómo ser invocado

METODOLOGÍAS DE MANEJO DE REQUISITOS BAJO EL ENFOQUE ORIENTADO A ASPECTOS

- La ingeniería de software orientada a aspectos –AOSD– proveen mecanismos para la identificación, modularización, representación y composición de intereses de corte transversal, tales como la seguridad, movilidad y restricciones de tiempo real.
- Las iniciativas del tratamiento de aspectos tempranos están enfocadas en el manejo de las propiedades de corte transversal en las etapas tempranas de la ingeniería de requisitos y el diseño de la arquitectura

Algunas aproximaciones al manejo de requisitos aspectuales

Utilización de un framework que permita realizar el estudio de cada una de las aproximaciones seleccionadas, el cual se basa en los siguientes criterios:

a) Fase

- ¿Qué fase del ciclo de vida es considerada por la metodología? Por ejemplo: análisis de requisitos, análisis del dominio y/o arquitectura.

b) Objetivo

- ¿Cuál es el objetivo de la metodología? Ejemplos son: identificación, especificación y/o evaluación de aspectos.

c) Artefactos

- ¿Cuáles son los artefactos adoptados que son usados para la identificación y/o especificación de aspectos tempranos?

d) Reglas heurísticas y procesos

- ¿La metodología usa reglas heurísticas explícitamente definidas o procesos para identificar y especificar aspectos tempranos?

e) Modelo de aplicación

- ¿Qué tipos de modelos/artefactos de aplicación es entregado?

f) Herramientas de soporte

- ¿Cualquier herramienta de soporte?

g) Soporte a la trazabilidad

- ¿En caso de que la herramienta soporte más de una fase del proceso de desarrollo, esto significa que son trazados/mapeados los artefactos o modelos?

Theme /Doc

- a) **Fase.** Análisis de requisitos.
- b) **Objetivo.** Identificar y especificar aspectos tempranos.
- c) **Artefactos.** Requisitos textuales, acciones, entidades y temas. Son distinguidos dos tipos de temas, temas base y temas de corte transversal.
- d) **Reglas heurísticas y procesos.** Consta de tres sub-procesos: identificación de acciones y entidad de los requisitos textuales (Identification of actions and entities from the textual requirements), categorización de acciones dentro de temas e identificación de temas de corte transversal. Cada sub-proceso incluye reglas heurísticas. La decisión si un tema es de corte transversal es explícitamente definida, mientras que las reglas para identificar acciones y entidades, y la decisión de cuáles acciones son suficientemente importantes para ser un tema permanece implícita (“Altamente intuitiva”).
- e) **Modelo de aplicación.** Provee vistas que exponen cuáles comportamientos son reubicados en requisitos. La ***vista de acción principal es usada para mostrar gráficamente la*** relación entre requisitos y acciones. La ***vista de acción de corte es usada para*** representar los temas de corte transversal. Junto a estas vistas, la ***vista de tema*** es usada para planear el diseño y modelado de los temas identificados. Las vistas de temas difieren de las vistas de acciones en que estas no solamente muestran requisitos y acciones, sino que también muestran elementos claves del sistema que necesitaran ser considerados para el diseño de cada tema.
- f) **Herramientas de soporte.** La herramienta Theme/Doc está bajo desarrollo. La herramienta toma requisitos textuales, concerns, grupos de concerns, entidades, *adjuntos, asociaciones y aplazamientos como entradas. Esta herramienta crea varias vistas y soporta la* identificación y especificación de concerns de corte transversal en la fase de análisis de requisitos.
- g) **Soporte a la trazabilidad.** Theme/Doc está enfatizado solamente en la fase de análisis de requisitos. Esta metodología no soporta la trazabilidad al análisis del dominio o la fase de diseño de la arquitectura.

Modularization and composition of aspectual requirements - AORE

- **Fase:** Análisis de requisitos.
- **Objetivo:** Identificar concerns de corte transversal al nivel de análisis de requisitos, especificando y evaluando estos concerns.
- **Artefactos:** Concerns, requisitos de los participantes, matriz de relación entre concerns y requisitos de los participantes, reglas de composición y una matriz de contribución.
- **Reglas heurísticas y procesos.** AORE provee los siguientes siete pasos del proceso que son ejecutados de manera secuencial: identificar y especificar concerns y requisitos de los participantes, identificar granularidad de las relaciones de los requisitos de los participantes, identificar aspectos candidatos, definir reglas de composición para un aspecto candidato y requisitos de los participantes, identificar y resolver conflictos entre aspectos candidatos, redefinir requisitos y especificar dimensión de los aspectos.
- **Modelo de aplicación.** El resultado del proceso es un conjunto de aspectos candidatos, los cuales son concerns que cruzan múltiples requisitos.
- **Herramienta de soporte.** ARCADE (Aspectual Requirements Composition and Decision). Esta herramienta está actualmente bajo desarrollo. La herramienta hace posible definir los puntos de vista de los requisitos, requisitos aspectuales y reglas de composición usando plantillas predefinidas.
- **Soporte a la trazabilidad.** AORE soporta la identificación y especificación en la fase de análisis de requisitos. No soporta la trazabilidad a otras fases del ciclo.

Integrating the NFR framework in a RE model - INFR

- **Fase:** Análisis de requisitos.
- **Objetivo:** Identificar concerns de corte transversal.
- **Artefactos:** Documentación de participantes, catálogos, plantillas, concerns, y puntos de corte. Las plantillas son usadas para especificar un concern. Un punto de encuentro especifica cuáles concerns de corte transversal deben ser integrados con un concern dado. Reglas de composición definen el orden en el cual los concerns serán aplicados en un punto de encuentro particular.
- **Reglas heurísticas y procesos.** La metodología INFR consiste de cuatro tareas: identificar concerns, especificar concerns, identificar concerns de corte transversal e integrar concerns.
- **Modelo de aplicación.** Aspectos candidatos, los cuales son concerns que atraviesan transversalmente múltiples requisitos.
- **Herramienta de soporte.** Actualmente esta metodología no dispone de una herramienta que la soporte.
- **Soporte a la trazabilidad.** La metodología INFR se enfoca en la fase de requisitos, y no tiene soporte a la trazabilidad a otras fases tempranas del ciclo de desarrollo de software.

COSMOS

- **Fase:** La metodología COSMOS abstrae el ciclo de vida del software. Por lo tanto este modelo puede ser usado en cada fase del ciclo de software.
- **Objetivo:** Modelar concerns y una plataforma para el desarrollo alternativo de modelado de concerns.
- **Artefactos:** El espacio de concerns consiste de: unidades, concerns, relaciones y restricciones. Unidades son elementos del modelo de concerns cuyo propósito es representar, en el modelo de concerns, los productos de trabajo del desarrollo de software. En el espacio de concerns, cada unidad hace referencia a un artefacto, el cual representa los detalles del software. En COSMOS nuevos tipos de artefactos, posiblemente envuelvan lenguajes no manejados previamente puede ser introducido. Esto incluye software (tales como requisitos, diseño o programación) lenguajes como también artefactos como directorios, archivos .JAR, XML y otros documentos.
- **Reglas heurísticas y procesos.** El modelo COSMOS no provee un proceso de soporte, reglas o heurísticas por medio de las cuales se pueda realizar la identificación y clasificación de los concerns y sus relaciones.
- **Modelo de aplicación.** COSMOS maneja un espacio de concerns en el cual se modelan las partes del software y los intereses de éste. Las partes del software pueden ser: un proyecto simple, un componente o un sistema completo.
- **Herramienta de soporte.** La herramienta Concern Manipulation Environment (CME) ofrece un conjunto de componentes para crear y componer concerns. Actualmente CME soporta identificación y modelado de concerns en un nivel detallado de diseño. Sin embargo, en el futuro la herramienta puede ser usada para identificación y modelado de concerns en todas las fases del ciclo de vida de software.
- **Soporte a la trazabilidad.** No hay un soporte a la trazabilidad explícito porque esta metodología abstrae el ciclo de vida del software.

	Fases del ciclo de desarrollo	Objetivo	Artefactos	Reglas heurísticas	Modelo de aplicación	Herramienta de soporte	Soporte a la trazabilidad
Theme/Doc	Análisis de requisitos	Identificar	Requisitos textuales, acciones, entidades y temas	Vistas de acción, soporte a reglas y soporte del proceso para identificación de aspectos.	Vista de tema	Actualmente en desarrollo	No
AORE	Análisis de requisitos	Identificar y especificar	Concerns, requisitos de los participantes, matriz relacionando concerns y requisitos de los participantes, una matriz de contribución	Reglas de composición, soporte del proceso para identificación y especificación de aspectos	Aspectos candidatos	ARCADE, actualmente en desarrollo	No
INFR	Análisis de requisitos	Identificar	Documentación, catálogos, plantillas, concerns y puntos de encuentro	Soporte del proceso, identificación de aspectos, reglas heurísticas y composición de reglas	Aspectos, candidatos, Modelo del sistema	Actualmente en desarrollo	No
COSMOS	Abstracto	Especificar aspectos	Concerns, relaciones, restricciones, unidades	-	Modelo de concerns	CME	No

Tabla 1. Evaluación de aproximaciones a aspectos tempranos. [25]

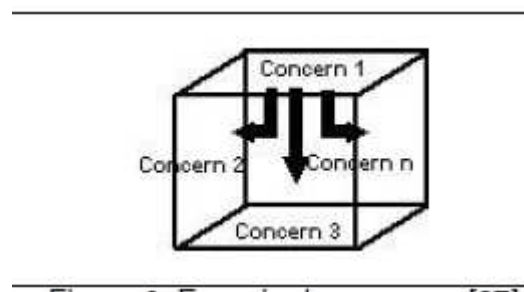
Problemas de las técnicas actuales

- El análisis de especificaciones de requerimientos no estructurados (no aprovechan la información adicional de un documento estructurado).
- La necesidad de ingresar información previa.
- El débil análisis sintáctico y la prácticamente ausencia de análisis semántico del texto.
- La falta de escalabilidad, principalmente cuando las herramientas son utilizadas en problemas reales y complejos.
- La poca o nula integrabilidad, trazabilidad y visualización provistas por los enfoques que no se apoyan en una metodología de modelamiento estándar.
- Los problemas de efectividad de los enfoques, al no tratar adecuadamente las ambigüedades y vaguezas del lenguaje natural. Como así también, no resolver adecuadamente el uso de sinónimos, o de palabras iguales con significados diferentes.

AORE

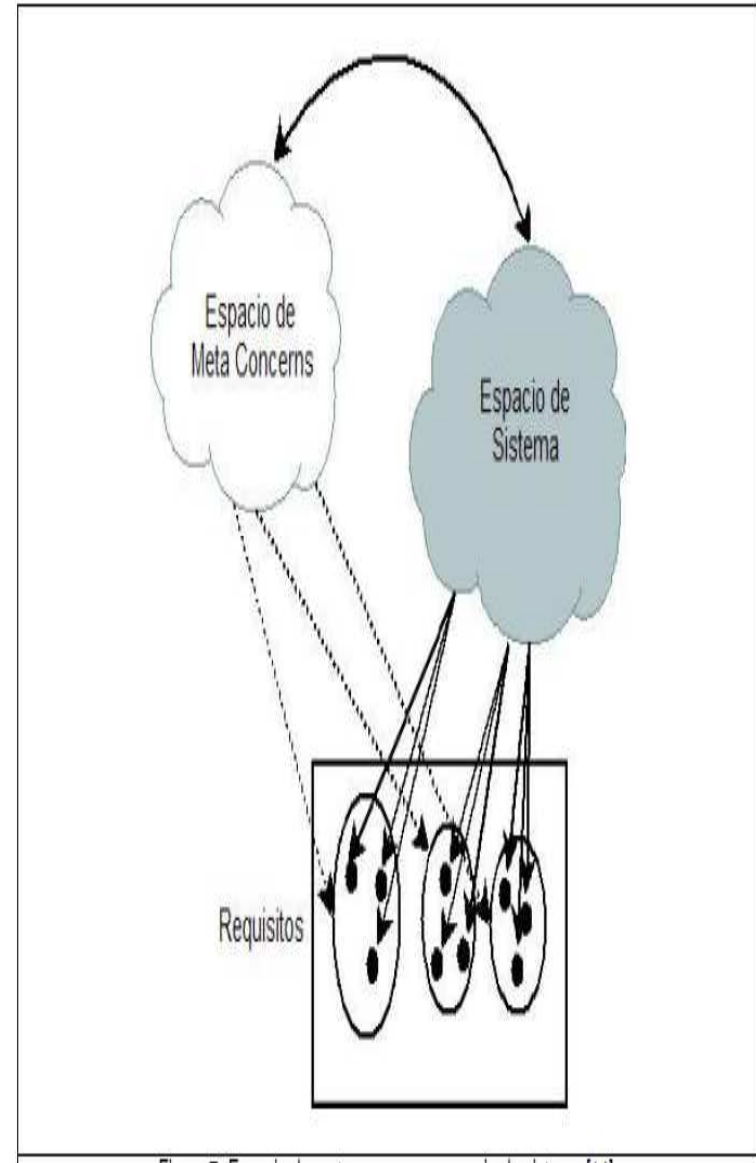
- El modelo AORE es un modelo propuesto por la Universidad Nova de Lisboa.
- Se propone un modelo que realice una descomposición de los requerimientos de una manera *uniforme, sin importar su naturaleza funcional, o no funcional*.
- Estudios posteriores de los autores muestran que el fenómeno de *crosscutting* se presenta no sólo en los requisitos no funcionales, sino también en los requisitos funcionales, sin embargo, otras aproximaciones a un modelo de ingeniería de requisitos basado en aspectos, no tienen en cuenta este hecho, por lo que los mecanismos ofrecidos por dichas aproximaciones, no permiten realizar un manejo adecuado de los requisitos funcionales, lo cual se refleja en la carencia en la identificación y caracterización de la *influencia de estos requisitos en otros concerns del sistema* [27].
- *Dado que AORE propone un tratamiento uniforme para manejar la descomposición de los requisitos, sin importar su naturaleza (funcional o no funcional), se hace posible proyectar la influencia de cualquier conjunto de requisitos en un rango de otros requisitos, soportando así una separación multidimensional de los concerns.*
- Una proyección específica la influencia de un concern en otros concerns y es realizada a través de las *reglas de composición* propuestas por el modelo.
- El modelo AORE propone un tratamiento *multidimensional de los concerns del sistema*. El enfoque multidimensional apunta a eliminar los problemas resultantes de utilizar una descomposición dominante, lo cual se logra a través del tratamiento uniforme de los diferentes tipos de requisitos en el sistema.
- El modelo soporta la detección temprana de conflictos y requisitos *sobrepuestos*, facilitando la negociación y la toma de decisiones entre los stakeholders.

- **El modelo AORE.** El modelo propuesto realiza un tratamiento uniforme de los concerns del sistema con el objetivo de poder determinar la influencia de cualquier conjunto de requerimientos en el sistema.
- Bajo este modelo, los concerns implican un *conjunto coherente de requisitos*.
- Sobre estos no se realiza ninguna clasificación, bien sea en casos de uso, puntos de vista o aspectos. Los concerns mantienen *encapsulados conjuntos coherentes* de requisitos tanto funcionales como no funcionales.
- Se percibe el espacio de los concerns, al nivel de requisitos, como un hipercubo.
- Cada cara del hipercubo representa un concern particular de interés

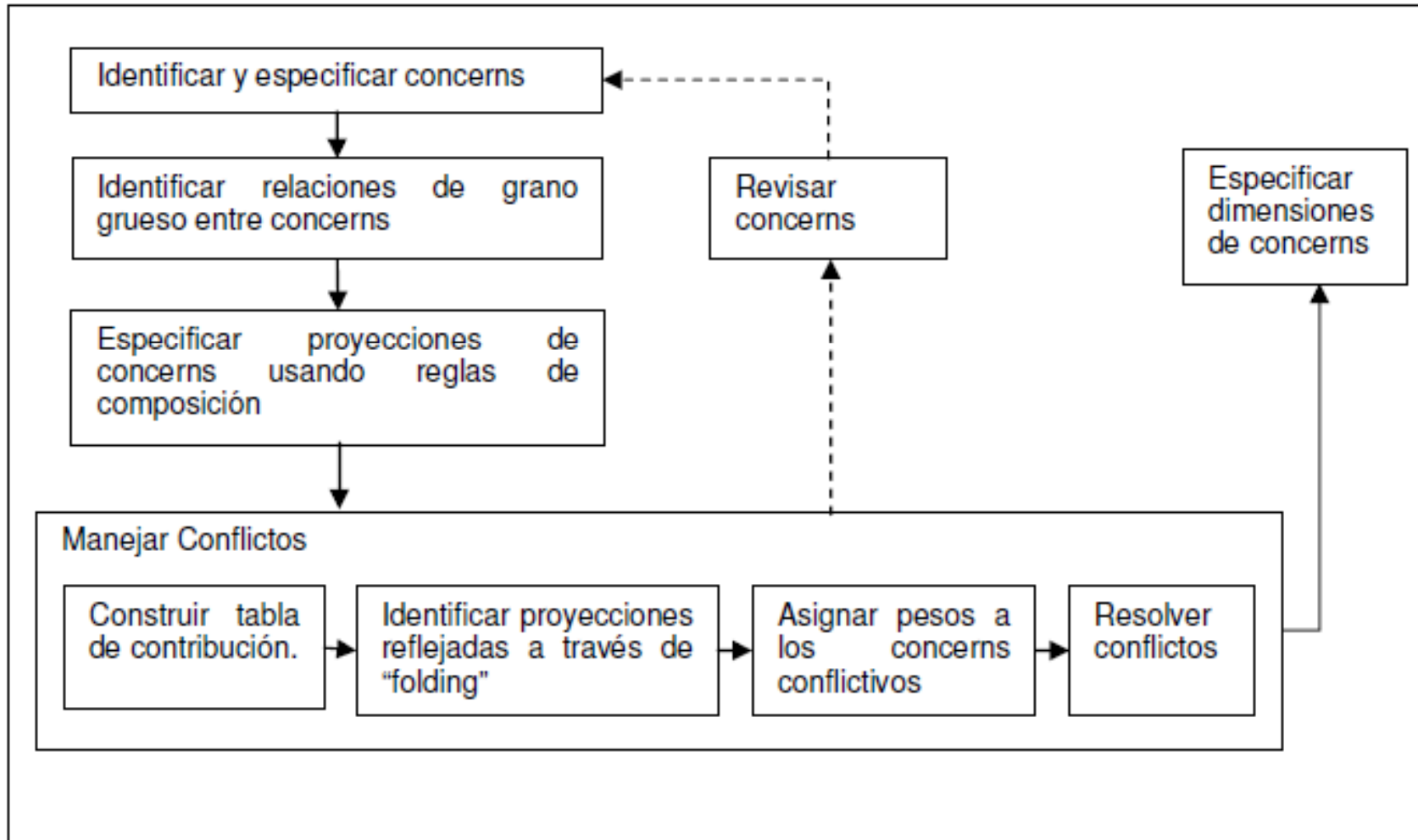


Espacio de meta-concerns y espacio de sistema

- La idea es proponer *catálogos de concerns*. Es decir, extender la noción de los catálogos a concerns funcionales, tales como facturación, registro, reservación y otros.
- A partir de esta observación, se divide el espacio de los concerns en dos espacios separados:
- **Espacio del sistema:** El cual comprende los diferentes tipos de sistemas que los desarrolladores pretenden realizar.
- **Espacio de meta concerns:** Comprende un conjunto abstracto de concerns funcionales y no funcionales, los cuales repetidamente se manifiestan en diferentes sistemas.
- A partir de los meta-concerns, cada sistema puede definir y especificar los concerns de su espacio de acuerdo a su dominio, al tiempo que permite iterativa e incrementalmente ampliar el espacio y definición de los meta-concerns.



Modelo de ingeniería de requisitos



Identificar y especificar concerns

- La identificación de concerns puede ser llevada a cabo a través de los mecanismos existentes de requisitos.
- El modelo proporciona plantillas basadas en XML para la identificación de los Concerns.

```
<?xml version="1.0" encoding="UTF-8"?>
<Concern name="Visitante">
  <Requirement id="1">
    El visitante podrá recuperar la información del
    sistema.
    <Requirement id="1.1">
      El visitante podrá acceder a la información sobre
      las atracciones.
    </Requirement>
    <Requirement id="1.2">
      El visitante podrá acceder información sobre su
      ubicación.
      <Requirement id="1.2.1">
        El visitante podrá validar la información
        sobre la situación si no corresponde a lo que
        él ve.
      </Requirement>
    </Requirement>
    <Requirement id="1.3">
      El visitante podrá obtener una lista de tours
      predeterminados que se encuentran disponibles.
    </Requirement>
  </Requirement>
  <Requirement id="2">
    El visitante podrá crear tours personalizados.
    <Requirement id="2.1">
      El visitante podrá especificar preferencias.
    </Requirement>
  </Requirement>
  <Requirement id="3">
    El visitante podrá seguir un tour.
    <Requirement id="1.3">
      El visitante podrá reconfigurar un tour.
    </Requirement>
  </Requirement>
  <Requirement id="4">
    El visitante podrá acceder a servicios externos.
    <Requirement id="4.1">
      El visitante podrá acceder a las reservaciones del
      hotel.
    </Requirement>
    <Requirement id="4.2">
      El visitante podrá acceder a las reservaciones de
      boletas de teatro.
    </Requirement>
  </Requirement>
</Concern>
```

Figura 9. Plantillas para identificación de concerns.

Identificar relaciones de grano grueso

- El siguiente paso en el modelo consiste en la identificación de las relaciones de grano grueso entre los concerns.
- Se relaciona cada concern con todos los otros concerns a través de una matriz. Una relación de grano grueso, es una relación en la cual no se detallan los requisitos que están implicados en la relación de los concerns.
- AORE propone diferentes técnicas para establecer las relaciones entre concerns, tales como análisis de dominio, ethnography, y procesamiento de lenguaje natural.
- La matriz permite observar cuáles concerns influyen a otros y en qué puntos existen influencias recíprocas o bidireccionales.

	Concern 1	Concern 2	...	Concern n
Concern 1		✓		
Concern 2				✓
...			...	
Concern n		✓		

Figura 10. Matriz de relaciones de grano grueso.

Especificar proyecciones de concerns

- Una vez se han identificado las relaciones de grano grueso entre los concerns, se procede a especificar las posibles proyecciones de cada concern en otros concerns.
- Para especificar las proyecciones se utilizan las regla de composición.
- Las reglas de composición, a diferencia de la relaciones de grano grueso, operan a la *granularidad de los requisitos individuales*. Esta *granularidad permite* especificar como un requisito dentro del concern, influye o restringe el comportamiento de un conjunto de requisitos de otros concerns. También se hace posible observar conflictos entre concerns a un nivel de *granularidad fina*.
- El hecho de poder observar conflictos a un nivel fino de granularidad es de gran importancia para el proceso de manejo de conflictos, ya que pueden evitarse *negociaciones innecesarias entre stakeholders para casos en los cuales pueden aparecer aparentes conflictos entre dos o más concerns, cuando en realidad son* requerimientos diferentes o aislados los que están siendo influenciados.
- De manera similar a la identificación de concerns, el modelo brinda unas plantillas por medio de las cuales se pueden especificar las proyecciones de concerns.

```
<?xml version="1.0" encoding="UTF-8"?>
<Composition>
  <Requirement concern="Visitor" id="all">
    <Constraint action="ensure" operation="during">
      <Requirement concern="Mobility" id="1"
        children="include" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>

  <Requirement concern="Visitor" id="all">
    <Constraint action="provide" operation="by">
      <Requirement concern="ElectronicDevice" id="all" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>
</Composition>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Composition>
  <Requirement concern="Mobility" id="1" children="include">
    <Constraint action="provide" operation="for">
      <Requirement concern="Visitor" id="all"/>
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>
```

```

  <Requirement concern="Mobility" id="1" children="exclude">
    <Constraint action="enforce" operation="for">
      <Requirement concern="Portability" id="1" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>

  <Requirement concern="Mobility" id="1" children="include">
    <Constraint action="affect" operation="on">
      <Requirement concern="Context" id="1"
        children="include" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>

  <Requirement concern="Mobility" id="2">
    <Constraint action="affect" operation="on">
      <Requirement concern="Availability" id="all" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>
</Composition>
```

Las reglas de composición se especifican a partir del tag *Composition*, el cual encapsula un conjunto coherente de reglas de composición.

El tag *Composition* está conformado por un conjunto de requisitos, los cuales tienen una estructura diferente a la presentada en la figura anterior. Si el requisito del concern tiene un sub-conjunto de requisitos, estos deben ser *incluidos o excluidos explícitamente en la restricción (Constraint) impuesta por un requisito del concern*, lo cual se logra con el uso del atributo “*children*”.

Por otra parte, si al atributo “*id*” del requisito se le asigna el valor “*all*”, indica que todos los requisitos especificados en el concern serán afectados por el *Constraint*, en otro caso, se debe especificar el identificador del requisito que se verá afectado.

El tag *Constraint* especifica una acción y un operador, los cuales definen cómo los requisitos del concern serán afectados por otro requisito.

El tag *Outcome* especifica el resultado después de afectar el requisito del concern con otro requisito. El valor de la acción describe si otro requisito del concern, o un conjunto de requisitos de éste, deben ser satisfechos o si simplemente la restricción especificada debe ser cumplida.

Las reglas de composición buscan establecer un lenguaje que permita definir la especificación de las relaciones, al tiempo que ofrece una estructura que puede ser *legible y entendible por los stakeholders*, lo cual es de gran importancia en la ingeniería de requisitos.

Las acciones
establecidos por el
modelo

Tipo	Descripción
enforce	Usado para imponer una condición adicional sobre un conjunto de requisitos del concern.
ensure	Usado para asegurar que una condición que debe existir para un conjunto de requisitos del concern actualmente existentes.
provide	Usado para especificar características adicionales a ser incorporadas para un conjunto de requerimientos del concern.
applied	Usado para describir reglas que aplican a un conjunto de requisitos del concern y deben alterar su resultado.
exclude	Usado para excluir algunos concerns o requisitos cuando el valor <i>all</i> es especificado.
affect	Usado para especificar que un conjunto de requisitos del concern alterarán el estado de otro concern.

Los operadores
establecidos por el
modelo

Tipo	Descripción
during	Describe el intervalo temporal durante el cual un conjunto de requisitos está siendo satisfecho.
between	Describe el intervalo temporal que transcurre entre la satisfacción de dos requisitos. El intervalo inicia cuando el primer requisito es satisfecho y termina cuando el segundo requisito empieza a satisfacerse.
on	Describe el punto temporal después de que un conjunto de requisitos han sido satisfechos.
for	Describe características adicionales que complementarán los requisitos del concern.
with	Describe una condición que se sostendrá para dos conjuntos de requisitos, uno dependiente del otro.
in	Describe una condición que se sostendrá para un conjunto de requisitos que han sido cumplidos.
AND, OR, XOR	Conjunción, disyunción y disyunción exclusiva.

Los valores de acción (Outcome)

Si se observan las palabras resaltadas, en la figura, es posible realizar una interpretación como se sigue:

“Todos los requisitos del Visitante deberán estar garantizados durante el requisito 1 de Movilidad, incluyendo sus hijos, esta composición garantiza que los requisitos del visitante están cumplidos”.

Tipo	Descripción
satisfied	Utilizado para asegurar que los diferentes puntos de vista de un requisito estarán satisfechos después de que las restricciones de un requisito de un concern han sido aplicadas.
fulfilled	Usado para asegurar que las restricciones de un requisito de un concern han sido realizadas con éxito.
observe	Usado para indicar que después de que se hayan aplicado los constraints, el resultado será observado para ver si la restricción del constraint se ha mantenido sobre la base o no.

```
<?xml version="1.0" encoding="UTF-8"?>
<Composition>
  <Requirement concern="Visitor" id="all">
    <Constraint action="ensure" operation="during">
      <Requirement concern="Mobility" id="1"
        children="include" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>

  <Requirement concern="Visitor" id="all">
    <Constraint action="provide" operation="by">
      <Requirement concern="ElectronicDevice" id="all" />
    </Constraint>
    <Outcome action="fulfilled" />
  </Requirement>
</Composition>
```

Figura 12. Interpretación de las reglas de composición por los stakeholders.

Manejar conflictos

- Teniendo especificadas las proyecciones de los concerns a través de las reglas de composición, es posible realizar la identificación y resolución de conflictos entre los concerns.
- El manejo de los conflictos se entiende como una macro-tarea del proceso que es comprendida por las siguientes actividades:
 - **Construir matriz de contribución**
 - **Folding**
 - **Asignar pesos**
 - **Resolver conflictos**

Construir matriz de contribución

- La construcción de la matriz de contribución consiste en identificar cómo cada concern contribuye con los otros concerns con los cuales se encuentra relacionado.
- Una contribución puede ser positiva (+), negativa (-) o bien, no tener contribución.
- La decisión acerca del tipo de contribución para cada caso particular es comúnmente difícil de tomar. Para esto puede recurrirse al uso de catálogos existentes de concerns o bien a la experiencia y al conocimiento empírico del dominio.

	Concern 1	Concern 2	...	Concern n
Concern 1		+		
Concern 2				-
...			...	+
Concern n		-		

Figura 13. Matriz de contribución entre concerns.

Folding

- El *folding* consiste en doblar la matriz de contribuciones a lo largo de la diagonal.
- Este permite obtener el *efecto acumulativo de las contribuciones para las* situaciones en las cuales dos concerns se influncian directamente el uno al otro.
- Al doblar la matriz, obtenemos las proyecciones reflejadas, las cuales dan la influencia combinada de un conjunto de requisitos en un concern particular.
- Al momento de realizar el doblado de la matriz, los concerns que tienen una proyección simétrica sobre otros, obtienen sus efectos acumulados.

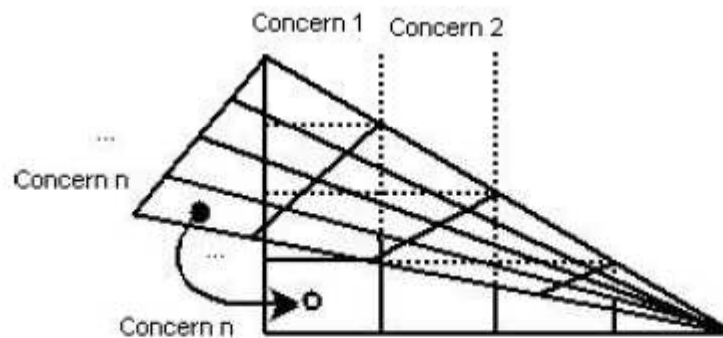


Figura 14. Tabla de contribuciones doblada por la diagonal.

	Concern 1		Concern 2		
Concern 1					
Concern 2	+			...	
...				...	Concern n
Concern n		-	+		

Figura 15. Proyecciones reflejadas.

Asignar pesos

- Ya identificados los concerns que tienen una contribución negativa con otros concerns, se procede a asignar pesos. Cada peso consiste en un número en el intervalo $[0 \dots 1]$ y representa la prioridad de un concern con respecto de los concerns en los cuales se proyecta.
- Los valores de los pesos son basados en las ideas de la lógica difusa y pueden ser interpretados de la siguiente manera:
 - Muy importante: Toma los valores en el intervalo $(0.8 \dots 1.0]$
 - Importante: Toma valores en el intervalo $(0.5 \dots 0.8]$
 - Medio: Toma valores en el intervalo $(0.3 \dots 0.5]$
 - Poco importante: Toma valores en el intervalo $(0.1 \dots 0.3]$
 - No importante: Toma valores en el intervalo $[0 \dots 0.1]$
- El uso de valores de la lógica difusa facilita a los stakeholders la tarea de atribuir los pesos a los concerns conflictivos. En caso de presentarse concerns conflictivos con el mismo peso, los stakeholders deben definir cuál de estos deberá ser rebajado para solucionar los conflictos. Los pesos se asignarán a los concerns con respecto al concern para el cual se especificó la regla de composición.

Resolver conflictos

- La solución de conflictos se realiza por medio de la asignación de prioridades. Ésta puede conllevar a una revisión de la especificación de requisitos (identificación de concerns y/o reglas de composición). Si esto sucede, entonces las proyecciones son revisadas y cualquier conflicto resultante puede ser resuelto. El ciclo debe repetirse hasta que todos los conflictos sean resueltos.

Especificar dimensiones de concerns

- La última actividad del modelo es la identificación de las dimensiones de los concerns.
- Los concerns en etapas tempranas pueden tener impacto en los artefactos de etapas posteriores del ciclo de desarrollo y que pueden ser descritas en términos de 2 dimensiones:
 - Mapping: Un concern se puede *mapear en una característica o función del sistema* (un método simple, un objeto o componente), decisión (decisión para la selección de arquitectura) o aspecto de diseño (y por tanto implementación). Se debe notar que a pesar de que algunos concerns pueden tener una naturaleza “crosscutting” en esta etapa, algunos de estos pueden no mapearse en un aspecto en etapas posteriores.
 - Influence: Un concern puede *influir diferentes puntos en un ciclo de desarrollo*, ejemplo: *disponibilidad influye la arquitectura del sistema, mientras que movilidad influye la especificación, la arquitectura, el diseño e implementación*.
- Se presenta un ejemplo de la especificación de las dimensiones de concerns.

Concern	Influence	Mapping
Visitante	Especificación, diseño, evolución	Función
Movilidad	Especificación, arquitectura, diseño, implementación	Aspecto
Disponibilidad	Arquitectura	Decisión
Contexto	Arquitectura, diseño, implementación	Aspecto

Tabla 5. Especificación de las dimensiones de concerns.

COSMOS

- El modelo COSMOS es un modelo propuesto IBM T.J. Watson Research Center.
- El modelo busca proveer una aproximación a la separación de concerns avanzada que esté enfocada a la aplicación multidimensional de vistas de concerns y que pueda ser aplicada a través del ciclo de vida del software. En esta sección presentaremos los elementos fundamentales que conforman el modelo y realizaremos una aplicación práctica de éste con el fin de entenderlo y analizarlo claramente.
- El modelo puede exhibir características relacionadas con la ocurrencia y distribución de concerns de diferentes dominios y tipos, concerns compartidos versus únicos, concerns explícitos versus implícitos.
- El modelo establece la forma en que los concerns son organizados, compartidos y vistos, al tiempo que exhibe sus características y múltiples dimensiones.
- La aplicación del modelado de concerns en etapas tempranas conlleva a un mejor entendimiento y un mejor tratamiento de los concerns y además pueden ayudar a identificar elementos reutilizables durante el análisis de requisitos. Esto facilita la formulación de vistas alternativas y representaciones de requisitos. El modelado de concerns de requisitos propuesto puede brindar un modelado más eficiente y sistemático de concerns y tecnologías orientadas a aspectos.
- Entre las motivaciones del modelo, encontramos que los aspectos pueden aparecer en cualquier etapa del ciclo de vida y en cualquiera de los artefactos que se generan en éste. Un mejor entendimiento de aspectos en requisitos puede ayudar a identificar principios orientados a aspectos y técnicas en el ciclo de vida, y proveer un uso más sistemático durante todo el ciclo de vida

El modelo COSMOS

- Es un esquema diseñado para el modelado multidimensional de espacios de concerns.
- Este comprende tres tipos de elementos, *concerns*, *relaciones* y *predicados*.
- *COSMOS divide los concerns básicos en dos categorías*, lógicos y físicos.
 - concerns lógicos como las “materias de interés” conceptuales en un sistema de software: discusiones, problemas, “ilities” (functionality, reliability, usability, efficiency, maintainability, portability), entre otras.

Se dividen en 5 categorías: ***classifications, classes, instances, properties y topics***
 - concerns físicos representan elementos de un sistema del “mundo real”, potencialmente se incluye software, hardware, sistemas y servicios. Estos pueden representar implementos, soporte u otros concerns que afecten los concerns lógicos.

Se dividen en las siguientes categorías: instances, collections, predicado y attributes.

Elementos del modelo

En primer lugar se tienen los concerns lógicos, los cuales pueden ser:

- **I) *Instances representa concerns específicos para el dominio del sistema.*** Representa la mínima expresión del concern.
- **II) *Classes concerns que son introducidos para categorizar otros concerns. Las classes pueden incluir otras classes y pueden ser utilizadas para clasificar instances lógicas, properties y los diversos tipos de concerns físicos.*** Los concerns o classes que hacen parte de una class no tienen que estar relacionados, el agrupamiento se realiza basado en características que son comunes a las intances, properties o classes que agrupa. Las classes en COSMOS no deben ser confundidas con las clases en notaciones de diseño o lenguajes de programación.
- **III) *Properties son concerns que caracterizan otros concerns. Ellos pueden ser*** aplicados a *classifications, classes o instances. Las properties aplicadas a classifications, aplican a las classes contenidas a las classifications; properties aplicadas a classes, son aplicadas a los miembros de la class.*
- **IV) *Topics son colecciones arbitrarias de concerns. Los topics capturan concerns*** relacionados a un tema, que pueden pertenecer a varias classes. *Los topics proveen una forma de organizar grupos de concerns que cruzan a través de otras categorizaciones de COSMOS. Mientras las classifications contienen classes de un tipo particular y las classes contienen subclases y miembros de un tipo particular, los topics pueden contener elementos de cualquier tipo.*
- **V) *Classifications representan sistemas de classes. Ellas están identificadas con*** una *class raíz y transitivamente se incluyen las subclases de esta. Desde una perspectiva de modelado de concerns, una clasificación corresponde a una dimensión en un espacio de concerns.*

Elementos del modelo

La siguiente clasificación corresponde a los concerns físicos, los cuales pueden ser:

- I) **Instances son elementos específicos de sistemas de software, incluyendo** productos particulares de trabajo, elementos particulares de software, sistemas específicos y servicios particulares. Las *instances pueden ser modeladas y analizadas independientemente, pero* no implica que sea requerido hacerlo. Las instancias físicas pueden ser incluidas en el análisis de las instancias lógicas.
- II) **Collections grupos de instances o collections físicas. Las collections en los** concerns físicos son análogas a los *topics en los concerns lógicos. Por ejemplo* paquetes de archivos fuentes y estaciones de trabajo en una red de área local. Las *collections pueden contener sub-collections y ser homogéneas o heterogéneas* con respecto a los tipos de *instances físicas incluidas*.
- III) **Attributes son propiedades específicas de instances o collections.** Se definen cuatro categorías de relaciones: ***categorical, interpretive, mapping y physical.*** El **esquema de COSMOS define tipos específicos de relaciones** *categorical*, mientras los tipos específicos de otras categorías deben ser *definidas* de acuerdo al dominio del sistema.
- IV) **Predicado y consistencia: El modelo incorpora varias condiciones de** consistencia relacionadas al esquema básico de integridad, algunos representan restricciones en las relaciones, por ejemplo, atributos aplican solo a *instances físicas y collections*. Otros predicados son *menos importantes* y pueden ser definidos como extensiones, por ejemplo, ¿puede un *instance* pertenecer a más de una *class*?, ¿puede un *instance* no pertenecer a una clase? Otros predicados pueden ser asociados con otras partes del esquema tales como relaciones *interpretive*, por ejemplo, *motivación implica contribución* (pero no viceversa).

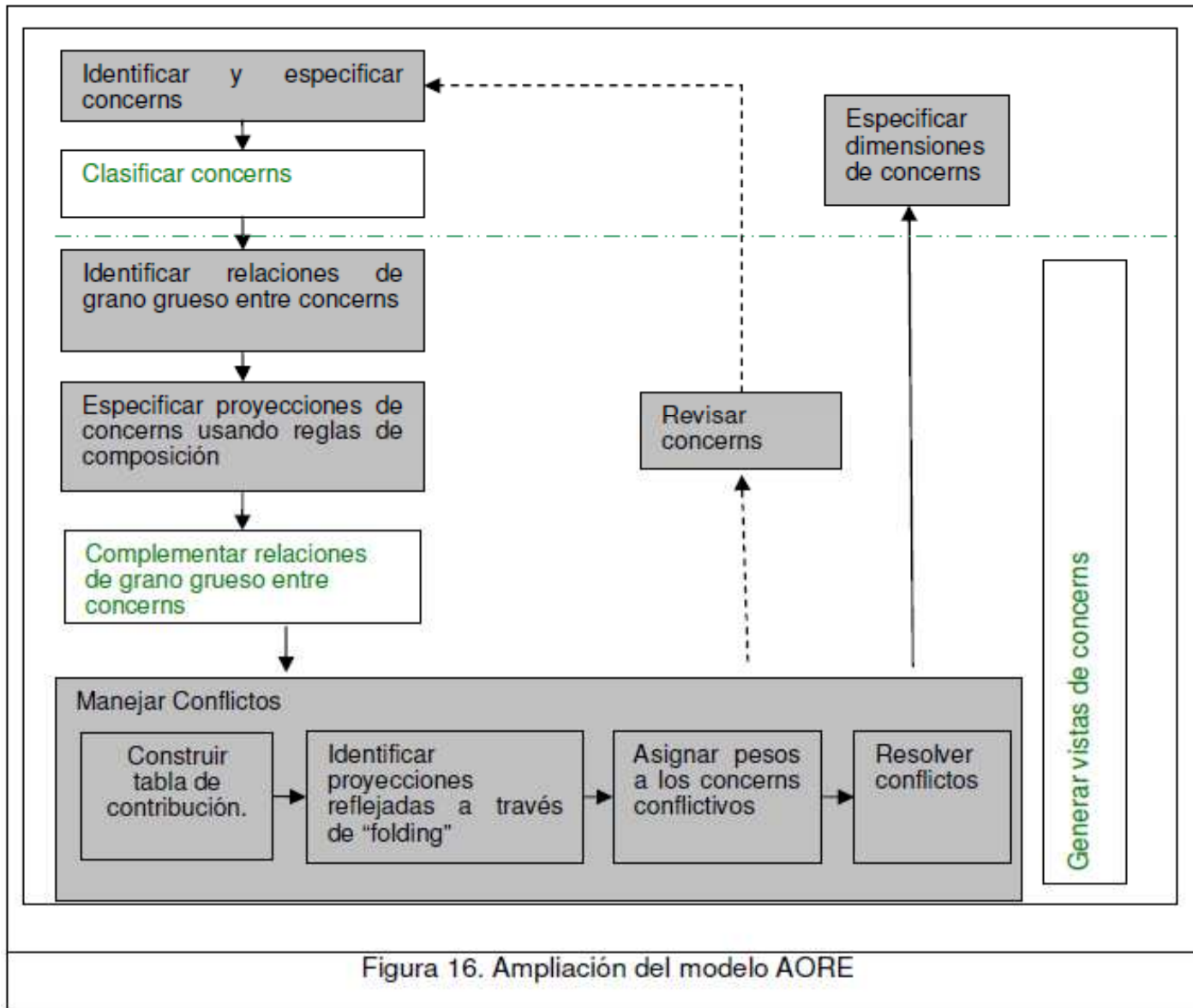
Las relaciones de tipo categórico sirven para relacionar concerns basado en sus categorías. Estas relaciones reflejan la semántica estructural de los tipos de concerns y define la integridad del modelo de concerns. COSMOS define siete subcategorías para las relaciones *categorical*.

- ***Classification relaciona classes a un sistema de classifications.***
 - ***Generalization relaciona classes en una relación de herencia.***
 - ***Instantiation relaciona instances a classes a las que pertenece. Las relaciones de tipo instantiation pueden ser tanto para concerns lógicos o físicos, pero dada una class puede contener sólo uno o el otro exclusivamente.***
 - ***Characterization relaciona properties a las classes o instances a las cuales se pueden aplicar.***
 - ***Membership relaciona instances físicas a collections a las cuales ellas corresponden.***
 - ***Attribution relaciona attributes a instances físicas o collections.***
 - ***Topicality relaciona concerns de cualquier tipo a un topic.***
-
- Las relaciones interpretativas (***interpretive***), ***reflejan asociaciones de*** interpretación semántica entre concerns lógicos. COSMOS no predefine tipos particulares de relaciones *interpretive*, *estas deben ser adicionadas de acuerdo a las necesidades de modelado de concerns.*
 - **Las relaciones físicas (*physical*), sirven para establecer relaciones entre** concern físicos.
 - **Las relaciones de mapeo (*mapping*), permiten relacionar concerns físicos y** lógicos. Por ejemplo *physical implementation es una relación mapping donde un concern físico implementa un concern lógico.*

CORE				Extensiones
Concerns	Classifications	Representan sistemas de clases y permite múltiples alternativas para clasificación de concerns.		Unidades de código pueden ser clasificadas de acuerdo al lenguaje de programación, representadas, operaciones contenidas, estructuras complejas.
Logical	Clases	Categorización de concerns.		Operaciones de responsabilidad, configurabilidad, componentes del diseño, comportamiento o estados.
	Instances	Representan concerns particulares, usualmente de alguna class.		Funciones específicas tales como adicionar un nuevo estado, como validar una entrada, especificar comportamiento, configurabilidad tales como configurabilidad del buffer.
	Properties	Son características que pueden atribuirse a clases.		Rendimiento, configurabilidad, robustez
	Topics	Grupos de concerns, generalmente de diferentes tipos de concerns, que son típicamente relacionados a un tema. Puede incluir clases, instancias y propiedades que están relacionados a un tema.		Manejo de errores, relaciones del consumidor.
	Collections	Grupos de concerns físicos. Las collections pueden contener sub-collections y ser homogéneas o heterogéneas con respecto a los tipos de instancias físicas incluidas.		Paquetes de archivos fuentes y estaciones de trabajo en una red de área local.
Physical	Instances	Son elementos específicos de sistemas de software.		Productos particulares de trabajo, elementos particulares de software, aplicaciones específicas y servicios particulares.
	Attributes	Valores particulares que caracterizan instancias físicas y collections. Generalmente estos reflejaran las propiedades de interés entre los concerns lógicos.		
	Classification	Relaciona classes a un sistema de clasificación.		
	Generalization	Relaciona classes en una relación de herencia.		Comportamiento de logging es una subclase de comportamiento.
	Instantiation	Relaciona instancias a classes a las cuales corresponden. Las relaciones de tipo instantiation pueden ser físicas o lógicas o físicas, pero dada una class puede contener sólo uno o el otro exclusivamente.		Adicionar un objeto es una instancia de de core-functionality.
Categorical	Characterization	Relaciona propiedades a las classes o instancias a las cuales se pueden aplicar.		Rendimiento es una propiedad de funcionalidad.
	Topicality	Relaciona concerns de cualquier tipo a un topic.		Garbage-collection configurability (una property) y garbage-collection algorithms (una class) son ambos un topic de garbage-collection.
	Membership	Relaciona instancias físicas a collections, a las cuales ellas corresponden.		Cache_core.java es un topic de cacheability, como el atributo de un libro es un atributo de un libro.
	Attribution	Relaciona atributos a instancias de tipo físico o collections.		Tamaño es un atributo de Cache_core.java.
	Significance			Admite, Contribuye a, lógicamente implementa, lógicamente parte de.
Mapping	Association			Afectado por, describe, modela, físicamente implementado por, representa.
	PhysicalRelation			Conecta, conecta a, físicamente afecta, físicamente parte de.
	<Sin subtipos definidos>			
	Predicados			

Tabla 8. Outline of the COSMOS concern-space modeling schema. Basada en [23] y [24]

Hacia un modelo integrado AORE - COSMOS



Elementos de ampliación del modelo

- **Clasificar concerns.** Esta actividad está orientada a proveer la clasificación de los concerns que componen un sistema y de la cual carece la propuesta inicial. Como se argumentaba anteriormente, la clasificación de los concerns está orientada a proveer vistas que permitan a cada stakeholder centrarse en los concerns que son de su interés.

Clasificación	Descripción	Motivación / Justificación
Objetivo	Objetivos del sistema, los cuales pueden estar agrupados o ser vistos de manera individual como un concern.	Propuesta realizada por el grupo de investigación de Ingeniería de Software y apartir de la experiencia en proyectos en la industria del software. Soportada en propuestas como VGraph [46]
Negocio	Basado en la definiciónb de concern propuesta por AORE, los concerns de negocio son una colección coherente de requisitos, los cuales pueden ser funcionales, de información y reglas de negocio.	Basados en la metodología propuesta por Amador Durán [39] y guiados por la necesidad de clasificar los concerns que plasman la funcionalidad principal de un sistema.
Riesgos	Son concerns que representan posibles dificultades a las cuales se expone el sistema en desarrollo.	Propuesta realizada por el grupo de investigación de Ingeniería de Software y apartir de la experiencia en proyectos en la industria del software.
Calidad	Generalmente son concerns asociados a características no funcionales del software ("ilities" ISO 9126) y requisitos no funcionales del sistema que se tenga en cuestión. Estas características pueden afectar todo el sistema o solo partes de éste.	Basados en la norma ISO 9126 [35] y en las propuestas de requisitos como NFR [47]
Físicos	Estos concerns comprenden un conjunto amplio de elementos que no son propios del sistema, pero los cuales pueden afectarlo considerablemente en cualquiera de las etapas. Estos concerns comprenden características tales como infraestructura, plataforma (Sistema Operativo), componentes externos, entorno de desarrollo, dispositivos, equipos y frameworks.	Motivados por la propuesta encontrada, estudiada y aplicada en el modelo COSMOS [9], [23] y [24].
Contexto	Este tipo de concerns comprende elementos del sistema que no son susceptibles a cambios y el sistema los debe considerar. Ejemplos de elementos que se consideran concerns de contexto son: restricciones de ley, servicios externos, sistemas existentes, modelos de referencia.	Propuesta realizada por el grupo de investigación de Ingeniería de Software e identificados en la experiencia en proyectos en la industria del software.

Tabla 14. Clasificaciones propuestas para concerns.

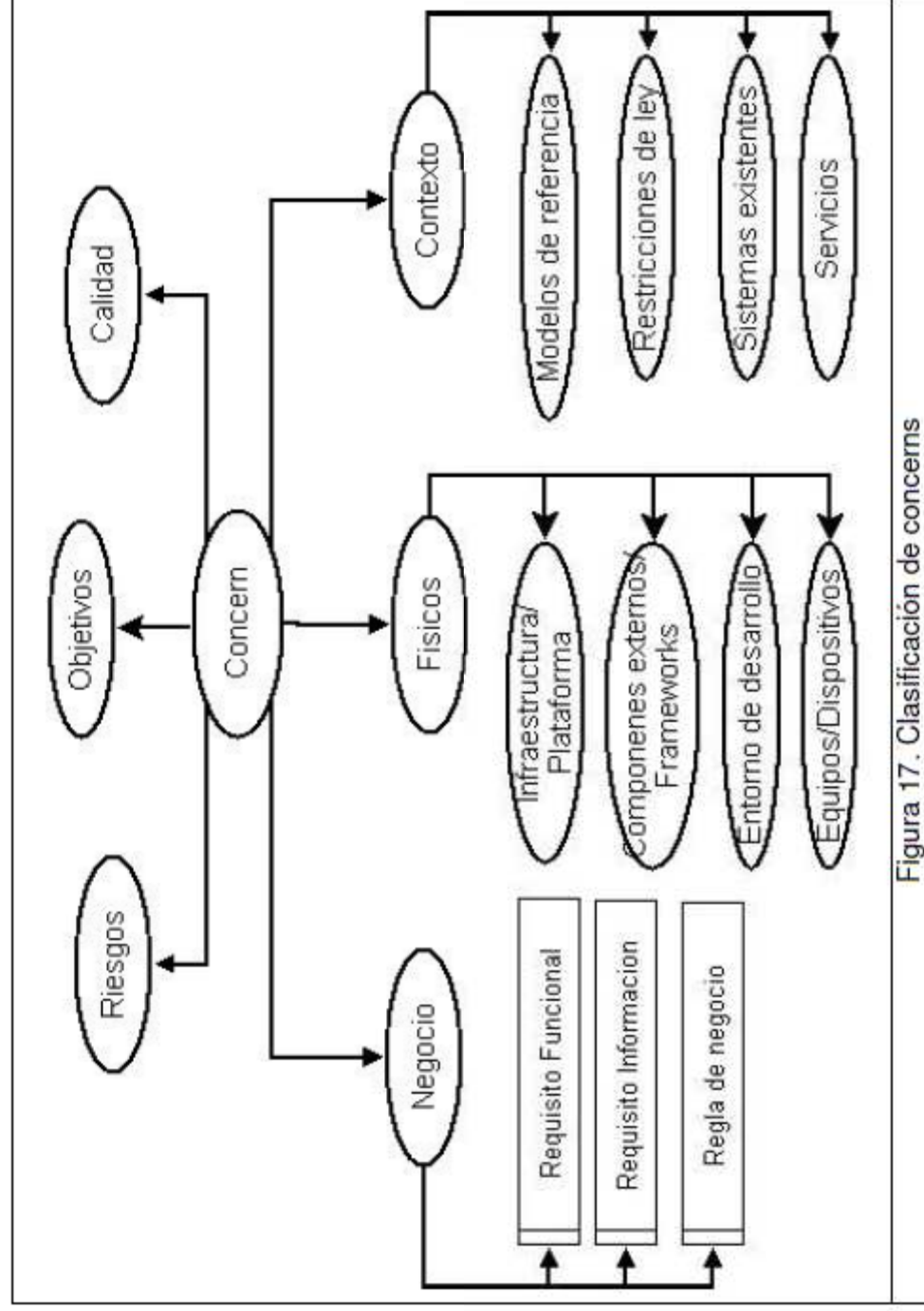


Figura 17. Clasificación de concerns

Complementar relaciones de grano grueso entre concerns

Relación	Descripción
Contribución	Un concern contribuye a otro cuando los requisitos del concern origen ayudan al cumplimiento de los requisitos del concern destino. Esta relación es vista como la relación recíproca a la afección. La contribución se da cuando el concern origen afecta de manera positiva al concern destino.
Restricción	Un concern restringe a otro cuando los requisitos del concern origen restringen los requisitos del concern destino.
Condición	Un concern condiciona a otro cuando los requisitos del concern origen condicionan los requisitos del concern destino. Se diferencia de la restricción porque las condiciones sólo se aplican bajo ciertas reglas o circunstancias.
Uso	Un concern usa a otro concern cuando los requisitos del concern origen usan requisitos del concern destino.
Extensión	Un concern extiende a otro concern cual los requisitos del concern origen extienden los requisitos del concern destino
Afección	Un concern afecta a otro concern cuando los requisitos del concern origen
	afectan los requisitos del concern destino. La afección es considerada como un aporte negativo entre los concerns. Esta relación se considera como la relación recíproca a la contribución.
Admisión	Un concern admite a otro concern cuando los requisitos del concern origen admiten la aplicación de los requisitos del concern destino.
Se aplica a	Un concern se aplica a otro concern cuando los requisitos del concern origen son aplicados a los requisitos del concern destino sin condicionarlos. Puede entenderse como la relación recíproca a la admisión.

Tabla 15. Relaciones de grano grueso

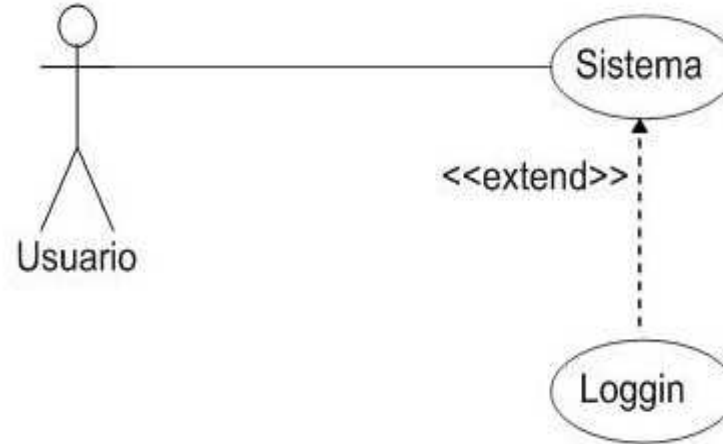
Generar vistas de concerns

Vista	Concerns	Descripción	Stakeholders
Negocio	Objetivos,	Vista orientada a mostrar los	Analistas.
	negocio, calidad y restricciones de ley	concerns que brinden la globalidad del sistema, reflejando la funcionalidad básica y las características de calidad que éste poseerá.	desarrolladores, SQA
Funcional	Objetivos, negocio.	Proporciona una visión de las funcionalidades de negocio que el sistema proveerá.	Usuario final.
QoS	Contexto y calidad.	Proporciona una visión de los elementos invariantes del sistema y características de calidad a tener en cuenta para el diseño del sistema.	Arquitectos, desarrolladores, SQA
Plataforma	Físicos, sistemas existentes y servicios.	Proporciona una visión de los concerns que determinan el entorno en el cual operará el sistema.	Operadores, integradores sistemas y arquitectos, desarrolladores
Proyecto	Objetivos, negocio, riesgos.	Proporciona una vista, de los objetivos del sistema, funcionalidades, riesgos y atributos de calidad que el sistema debe satisfacer.	Gerentes de proyecto, SQA.

Tabla 16. Vistas de concerns.

DIAGRAMAS DE CASOS DE USO

- Diagramas de Casos de Uso son fundamentales para la identificación de requerimientos en el DSOO. Como DSOA es una extensión de DSOO, es posible la aplicación de Diagramas de Casos de Uso en DSOA (Jacobson et al., 2004).
- Se debe considerar que Diagramas de Casos de Uso no representan solamente una técnica asociada a requerimientos.
- Diagramas de Casos de Uso son una técnica de Ingeniería de Software para dirigir completamente el ciclo de vida del desarrollo de software, donde es relevante la identificación de Incumbencias para su modelación, implementación y validación.
- Justamente, Diagramas de Casos de Uso permiten expresar Incumbencias de usuario, y proveer una temprana validación de las mismas.
- De esta forma, Diagramas de Casos de Uso UML son un útil medio para dirigir el DSOA, donde el desarrollo incremental mediante Casos de Usos representa un tradicional enfoque de desarrollo de software.



En el diagrama existe una modelación de Incumbencia cruzada, es claro que Diagramas de Casos de Uso UML permiten la capturar temprana efectiva de Incumbencias desde los requerimientos de usuario.

Como señala (Jacobson et al., 2004), con el uso Diagramas de Casos de Uso es posible conocer y avanzar en la comprensión del comportamiento deseado del sistema ante las potenciales acciones del usuario y las posibles respuestas del sistema.

Una idea adicional que será utilizada en la modelación de caso de estudio es que la representación de relaciones de inclusión (incluye) usando relaciones de extensión para soportar completamente el paradigma de DSOA.

También es posible la utilización de relaciones de herencia (Jacobson et al., 2004; Pressman, 2001), las cuales también pueden ser modeladas como relaciones de extensión.

ANÁLISIS SEMÁNTICO DE ASPECTOS PARA LA IDENTIFICACIÓN DE ASPECTOS



- Técnica de identificación de aspectos candidatos dada la especificación de requerimientos.
- Desarrollada con el objetivo de solucionar los problemas de las propuestas ya existentes, mejorando principalmente la precisión de la identificación.
- El proceso propuesto define una técnica automatizada que resuelve los defectos detectados, generalmente causados por las ambigüedades y vaguezas del texto natural.
- La técnica utiliza estrategias:
 - Análisis del texto con procesadores de lenguaje natural
 - Utilización de algoritmos de desambigüación semántica
 - Análisis de las sentencias utilizando patrones sintácticos
 - Explotación de relaciones semánticas para agrupar palabras semánticamente relacionadas, generación de estructuras de navegación de los concerns (para realizar la identificación).
 - Filtrado de concerns
 - Ponderación de los concerns según su importancia.

Problemas de las técnicas actuales

- El análisis de especificaciones de requerimientos no estructurados (no aprovechan la información adicional de un documento estructurado).
- La necesidad de ingresar información previa.
- El débil análisis sintáctico y la prácticamente ausencia de análisis semántico del texto.
- La falta de escalabilidad, principalmente cuando las herramientas son utilizadas en problemas reales y complejos.
- La poca o nula integrabilidad, trazabilidad y visualización provistas por los enfoques que no se apoyan en una metodología de modelamiento estándar.
- Los problemas de efectividad de los enfoques, al no tratar adecuadamente las ambigüedades y vaguezas del lenguaje natural. Como así también, no resolver adecuadamente el uso de sinónimos, o de palabras iguales con significados diferentes.

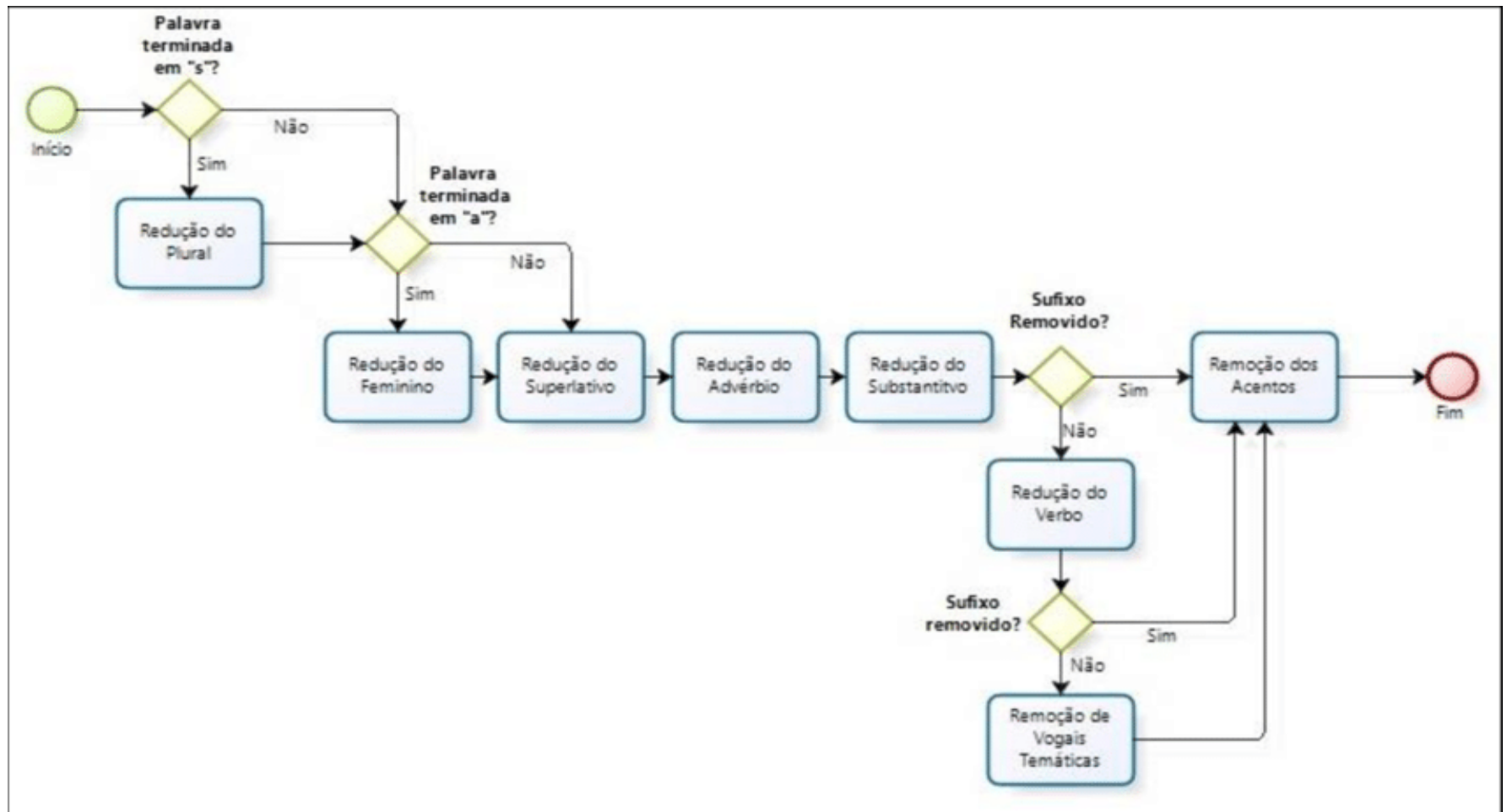
Identificación automatizada de aspectos

- I. Aprovechar la información provista por una documentación estructurada para mejorar la identificación de aspectos candidatos.
- II. Incorporar la técnica a alguna metodología que se integre al proceso de desarrollo del sistema de manera tal permitir tener una mayor visibilidad que al utilizar un enfoque genérico (que no detalla estas características), y que al mismo tiempo, provea de un conjunto de vistas que faciliten la lectura de los concerns.
- III. Evitar que el desarrollador deba ingresar información adicional para la realización del análisis para aumentar tanto la velocidad de la aplicación como la transparencia.
- IV. Proveer un análisis léxico, sintáctico y semántico del texto de la documentación, que aumente la efectividad y reduzca los problemas causados por las ambigüedades del lenguaje natural.
- V. Incluir una variedad de filtros que permitan reducir la información generada por el análisis hasta obtener una cantidad manejable por el analista (para cuando éste deba intervenir manualmente en la selección).

Para cumplir ...

- Utilizar un *Procesador de Lenguaje Natural* (*Natural Language Processor, NLP*) que realice el análisis léxico y sintáctico del texto. Este análisis etiqueta las palabras de cada sentencia (indicando si es verbo, sustantivo, etc.).
- Realizar un análisis semántico de cada palabra en el que se determine precisamente qué sentido se le da a la palabra según el contexto que la rodea.
- Realizar un análisis de las palabras que permita agrupar aquellas identificadas con significados semánticos similares, utilizando diccionarios semánticos. También son tenidos en cuenta diccionarios de sinónimos y algoritmos de stemming.
- Contar con información estadística de cada una de las palabras, para poder establecer la relevancia de las mismas.

Ej.: algoritmos de stemming



1.- Desarrollo de la técnica de identificación de aspectos

- La mayoría de los análisis de identificación de aspectos se basan en la búsqueda de verbos para identificar comportamiento crosscutting.
- Sería más adecuado y preciso identificar sobre qué objetos actúan estas acciones o verbos, para aumentar la efectividad del enfoque. (debido a que una misma acción o verbo, al ser aplicada sobre diferentes objetos, puede llegar a considerarse como diferentes comportamientos).
- Al ser mayor la cantidad de información que se tiene de cada concern crosscutting potencial, la identificación de pares (verbo, objeto directo) conlleva a una mayor efectividad de análisis.
- El algoritmo principal realiza un análisis basándose en un esquema similar al de los grafos AOIG, buscando pares (verbo, objeto directo) para identificar las acciones clave.
- Aunque como en el texto natural no siempre se incluye información sobre el objeto al que afecta el verbo, es necesario también considerar los verbos solos (sin objeto directo), para no perder información.
- Al mismo tiempo, el uso de una estructura similar al grafo AOIG permite realizar diferentes tipos de análisis sobre los concerns, debido a la gran cantidad de información que provee.

1.- Desarrollo de la técnica de identificación de aspectos

- Utiliza UML como base de la aplicación de identificación de aspectos. (ventajas: trazabilidad, integrabilidad, extensibilidad mediante “profiles”, se permite especificar los aspectos en un desarrollo de sistemas integrando los aspectos de manera transparente en el proceso de desarrollo).
- Se aprovecha el formato estructurado de las especificaciones de los caso de uso para identificar aquel comportamiento crosscutting que deba ser considerado funcional o no funcional.
- Se considera como no funcional cada verbo y objeto directo que se encuentre en la sección requerimientos “no funcional” de la especificación de casos de uso o en cualquier documento de requerimientos suplementarios, mientras que en el resto de las especificaciones de caso de uso, como el flujo básico, alternativo, condiciones, etc. será considerado como funcional.
- El análisis del algoritmo se realiza en dos partes separadas y consiste principalmente del etiquetado de parte del discurso (usando un NLP) y la desambigüación semántica de las palabras.
 - 1) Se realiza sobre los textos de los flujos básicos y alternativos de los casos de uso.
 - 2) Se realiza sobre los requerimientos no funcionales del sistema y sobre los requerimientos adicionales de cada una de las especificaciones.
- Esta diferenciación se realiza para poder tener noción de cuándo el par encontrado está relacionado con un concern funcional y otro no funcional.

1.- Desarrollo de la técnica de identificación de aspectos

- Para llevar a cabo la desambiguación semántica, se utilizan bases de datos léxicas. (cuando el dominio es desconocido, estas BBDD se comportan estadísticamente de manera estable)
- Se construye un grafo que muestra la utilización de los nodos (verbo, objeto directo) y nodos (verbo), pero en vez de hacerlo directamente a su nodo de uso en la especificación se agrega un nodo intermedio que agrupa pares de “verbos semánticamente relacionados” con “sustantivos semánticamente relacionados”.
- Esta agrupación de palabras tiene su origen en la navegación de las relaciones semánticas entre términos de la base de datos semántica, en la que se buscan términos similares.
- Estos agrupamientos, al estar realizados posteriormente a la desambiguación semántica, resuelven los problemas de sinónimos, ambigüedades y vaguezas del lenguaje natural, aumentando de manera notoria la efectividad del enfoque.

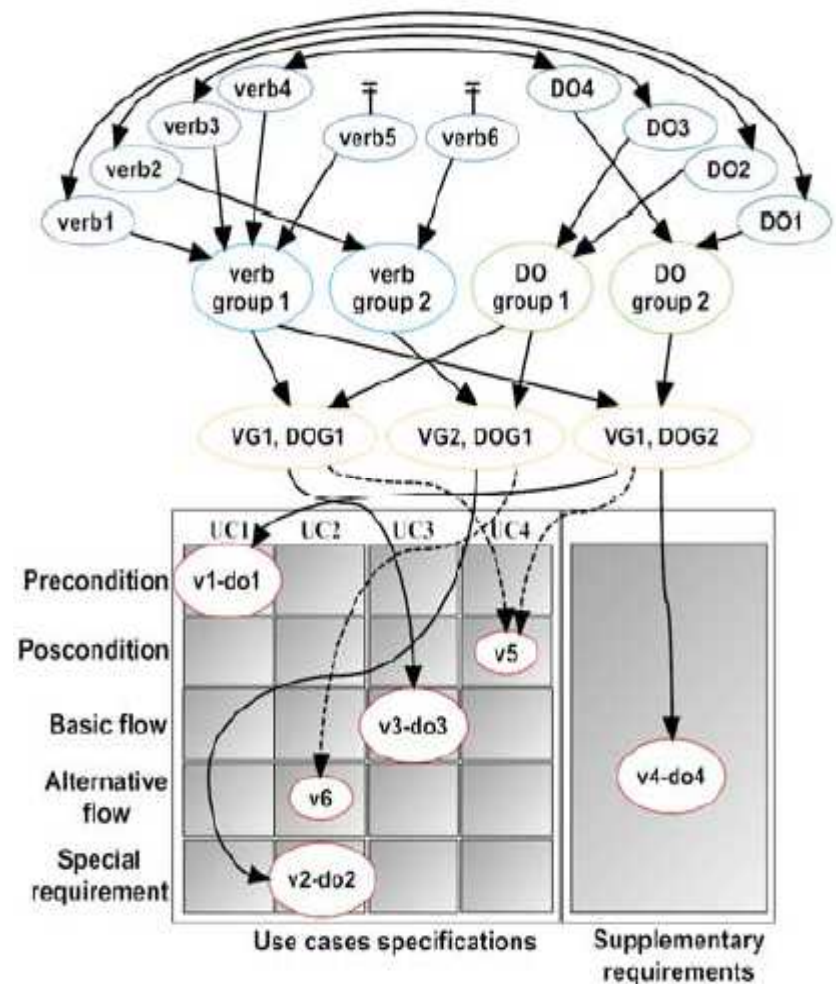


Ilustración 1 - Grafo de identificación de aspectos

1.- Desarrollo de la técnica de identificación de aspectos

- Luego de la creación del grafo, se realiza un simple recorrido del mismo para determinar aspectos potenciales, teniendo en cuenta sus usos en más de una especificación de casos de uso.
- Cada uno de estos aspectos potenciales es clasificado en funcional y no funcional según el verbo del cual forma el par.
- El resultado obtenido es ordenado realizando un ranking en el que cada uno de los nodos es ponderado por las estadísticas generadas en el análisis del texto previamente efectuado más la información de crosscutting que es obtenida del recorrido del grafo.
- La lista ordenada es presentada al analista para que seleccione y determine los aspectos a considerar.
- Se pueden utilizar filtros para reducir la lista según criterios preestablecidos, basándose en la estructura del grafo de identificación para efectuar el filtrado, lo cual permite una mayor escalabilidad del enfoque. Entre los criterios de filtrado, es posible encontrar los filtros de verbo, de objetos, etc.

2. Proceso de identificación de aspectos candidatos

- El proceso consta principalmente de tres bloques diferentes:
 - *Análisis del procesador de lenguaje natural (NLP Analysis)*
 - *Generación del grafo (Graph Generation)*
 - *Buscador de aspectos candidatos (Aspect Finder).*

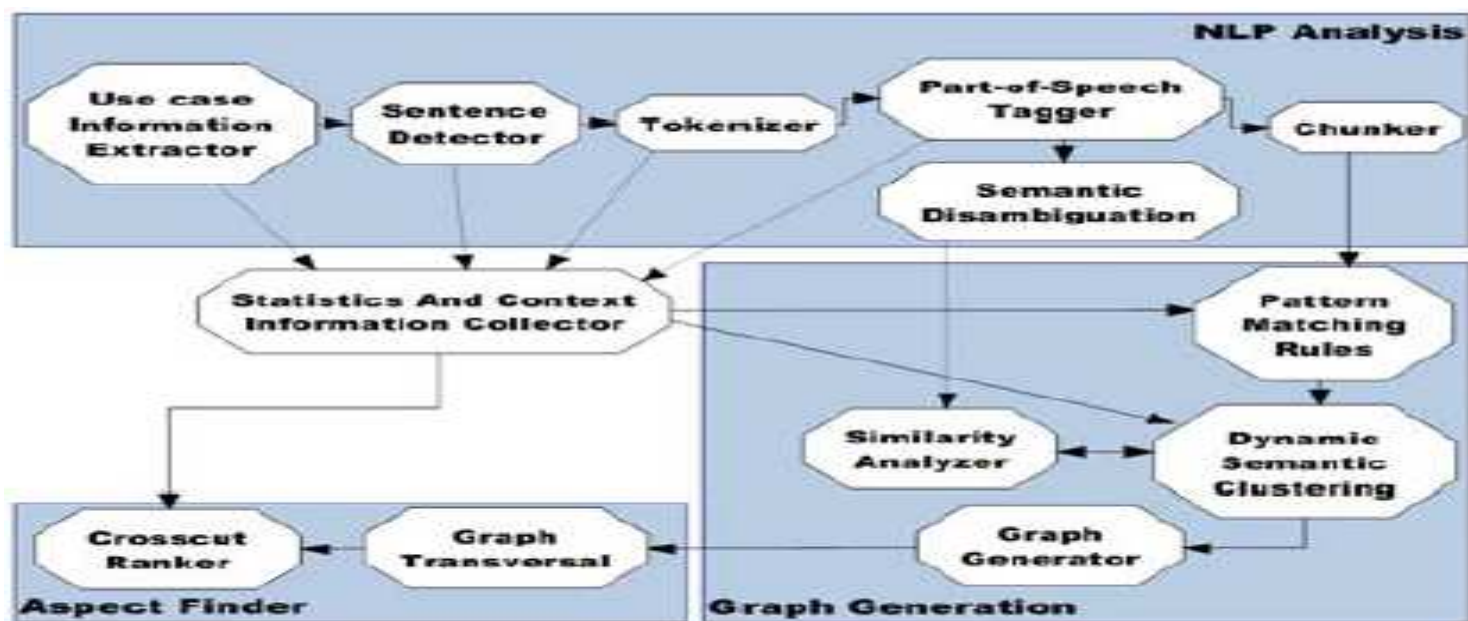


Ilustración 2 - Proceso de identificación

2.1. NLP Analysis

- Tiene como finalidad llevar a cabo un análisis léxico, sintáctico, y semántico del texto de las especificaciones, así como también efectuar un estudio estadístico y de ubicación de este texto.
- Consta de seis tareas explícitas que llevan a cabo sus objetivos:
 - extracción de información de los casos de uso (Use case Information Extractor)
 - separación de sentencias (Sentence Detector),
 - separación de palabras (Tokenizer),
 - etiquetado POS (Part-of-Speech Tagger),
 - desambigüación semántica (Semantic Disambiguation)
 - agrupamiento en grupos sintácticos (Chunker).
- Todas las tareas se realizan con el procesador NLP, excepto la tarea Semantic Disambiguation, que se ocupa de desambigüar el significado de las palabras según el contexto que la rodea. (es necesario para tratar de solucionar los problemas de ambigüedad y vaguezas propios de los lenguajes naturales).



2.1. NLP Analysis

- Para realizar la desambigüación de los sentidos de la palabra se utiliza una modificación al algoritmo (*“Maximum Relatedness Disambiguation Algorithm”*), que utiliza métricas de relaciones semánticas.
- Este algoritmo desambigua una palabra polisémica eligiendo aquel sentido de la palabra que maximiza su relación con aquellas palabras cercanas que estén dentro de la ventana de contexto.
- El algoritmo fue modificado para que se aplique a todas las palabras del texto y con el tamaño de ventana que abarca la oración donde se encontró la palabra objetivo.
- Las medidas utilizadas para determinar cuánto se relacionan las palabras en el algoritmo son dos:
 - Propuesta rápida pero menos efectiva (Lesk Gloss Overlap Measure)
 - Más efectiva y más costosa en velocidad (Pedersen y otros (Lesk Extended Gloss Overlap Measure)).

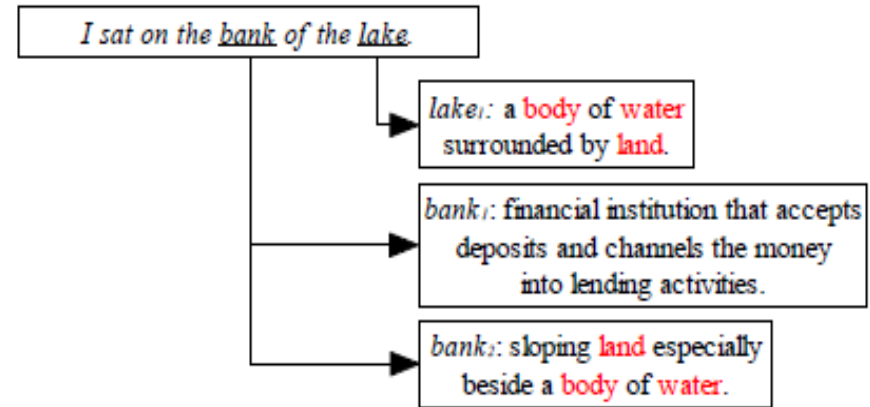


Ilustración 3 - Medida de Lesk

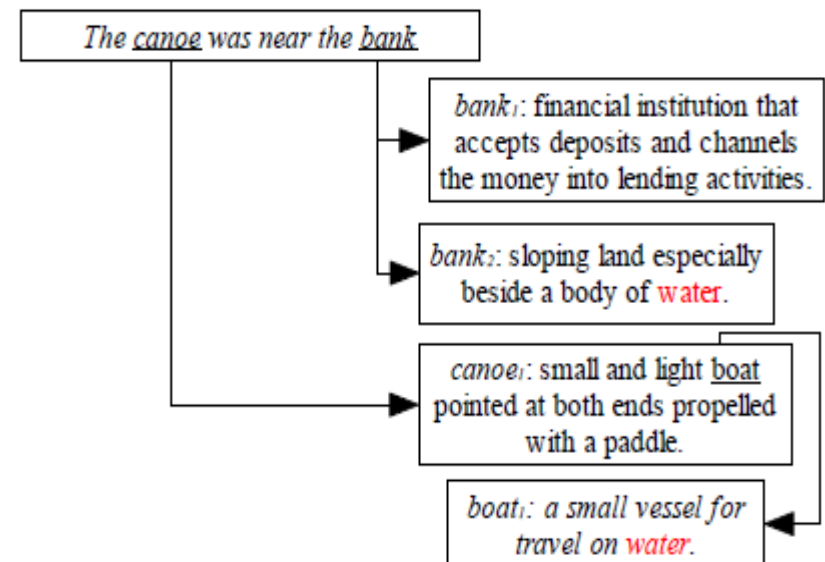


Ilustración 4 - Medida de Lesk extendida

2.1. NLP Analysis

- En el algoritmo propuesto, el inventario de donde se obtienen las definiciones se corresponden con la base de datos semántica WordNet.
- Ésta es una base de datos léxica que contiene información acerca de sustantivos, verbos, adjetivos y adverbios. El objetivo es organizar los conceptos relacionados en *conjuntos de sinónimos (Synonym Set, Synsets)*.
- *Cada synset puede considerarse como la representación de un concepto o sentido (Sense).*
- *WordNet conecta los conceptos mediante una variedad de relaciones.*
- *Como consecuencia se crea una red donde los conceptos relacionados pueden ser identificados mediante el cálculo de una distancia relativa entre ambos.*

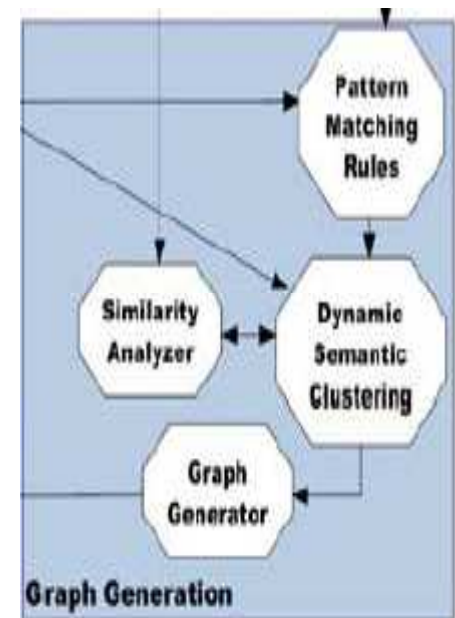
- **WordNet** es una base de datos léxica del Idioma inglés que agrupa palabras en inglés en conjuntos de sinónimos llamados *synsets*, proporcionando definiciones cortas y generales y almacenando las relaciones semánticas entre los conjuntos de sinónimos.
- Propósito es doble: producir una combinación de diccionario y tesauro cuyo uso sea más intuitivo, y soportar los análisis automáticos de texto y las aplicaciones de inteligencia artificial.
- “lexicón computacional” más usado para desambiguar el significado de las palabras, una tarea que tiene como objetivo asignar el concepto más apropiado a los términos en contexto.
- La base de datos y las herramientas del software se han liberado bajo una licencia BSD y pueden ser descargadas y usadas libremente; además, la base de datos puede consultarse en línea.

Structure of Wordnet						
Semantic relation	Description	Part of speech				Example
		N	V	Adj	Adv	
Synonym	A concept that means exactly or nearly the same as another. <i>WordNet</i> considers immediate hypernyms to be synonyms.	×	×	×	×	{ <i>sofa, couch, lounge</i> } are all synonyms of one another. { <i>seat</i> } is the immediate hypernym of the synset.
Antonym	A concept opposite in meaning to another.	×	×	×	×	{ <i>love</i> } is the antonym of { <i>hate, detest</i> }.
Hypernym	A concept whose meaning denotes a superordinate.	×	×			A { <i>feline, felid</i> } is a hypernym of { <i>cat, true cat</i> }.
Hyponym	A concept whose meaning denotes a subordinate.	×	×			A { <i>wildcat</i> } is a hyponym of { <i>cat, true cat</i> }.
Substance meronym	A concept that is a substance of another concept.	×				A { <i>snowflake, flake</i> } is substance of { <i>snow</i> }.

Link: <https://wordnet.princeton.edu/>
<https://www.lexicoweb.com/>
<http://grial.uab.es/recursos/wordnet30.php>

2.2. Graph Generation

- Propósito: realización del grafo de identificación de aspectos propuesto llevando a cabo aquellos pasos previos necesarios para la construcción del mismo.
- El bloque está formado por cuatro tareas que ejecutan esta elaboración:
 - *Detección de verbos y objetos directos (Pattern Matching Rules)*
 - *Análisis de similitud (Similarity Analyzer)*
 - *Agrupamiento dinámico de verbos y sustantivos (Dynamic Semantic Clustering)*
 - *Creación del grafo (Graph Generator)*



2.2. Graph Generation

Tarea *Pattern Matching Rules*

- *Se ocupa de llevar a cabo la identificación de los verbos y los objetos directos en las sentencias del texto, tomando como entrada la salida del agrupador sintáctico.*
- *Se aplican una serie de reglas que permiten identificar los verbos buscando el verbo dentro de las frases verbales e identificando en las frases sustantivas posteriores el objeto directo.*
- *Para realizarlo, se determina el tipo de verbo que se detectó en la frase verbal y en base a ello se identifica el patrón sintáctico que le corresponde.*
- *De esta manera es posible saber si la frase contiene o no un objeto directo, y si existe, en que lugar se encuentra.*
- *Se identifican pares Verbo-Objeto directo en aquellas frases que cuenten con objeto directo, y solamente el simple Verbo en aquellas que no tengan uno.*

2.2. Graph Generation

Tarea Similarity Analyzer

- *Encargada de realizar el análisis de similitud entre verbos (y objetos directos) para posteriormente agruparlos en grupos semánticamente relacionados.*
- *Está basado en el resultado de la desambigüación semántica realizada previamente que tiene como salida la determinación del sentido o significado correcto de los términos según su contexto.*
- *Dados dos términos (verbos u objetos directos), se establece la métrica de similitud entre éstos utilizando las relaciones semánticas provistas por el inventario de sentidos, navegando estas relaciones en busca de las palabras que representan los synsets navegados, teniendo como origen del recorrido el sentido desambigüado de cada término.*
- *La forma de establecer el puntaje es contando el número de términos que tienen en común los dos conceptos, considerando estos dos casos:*
 - *Si se comparan dos verbos, se usaran dos de las jerarquías provistas por WordNet para ampliar las palabras que representan los verbos: *Hypernyms*, donde el verbo Y es un *hypernym* del verbo X, si la acción X es un tipo de Y (“travel” y “movement”); y *Troponyms*, donde el verbo Y es un *troponym* del verbo X, si la acción Y es realizar X de alguna forma (“to lisp” y “to talk”).*
 - *Si se comparan dos sustantivos (objetos directos), se utilizaran dos de las jerarquías provistas por WordNet para ampliar las palabras que representan los sustantivos: *Hypernyms*, donde Y es un *hypernym* de X, si cada X es un tipo de Y (“canine” y “dog”); y *Hyponyms*, donde Y es un *hyponym* de X, si cada Y es un tipo de X (“dog” y “canine”).*

- Esto quiere decir que para cada término que se compara, se realiza una explosión de cada palabra para poder tener un mejor entendimiento del concepto que representa el término. Para cada uno de los dos términos que se deben comparar, se realiza la explosión (está controlada a no más de tres niveles en cualquier dirección) y se almacenan las palabras que representan a cada synset alcanzado por la explosión.

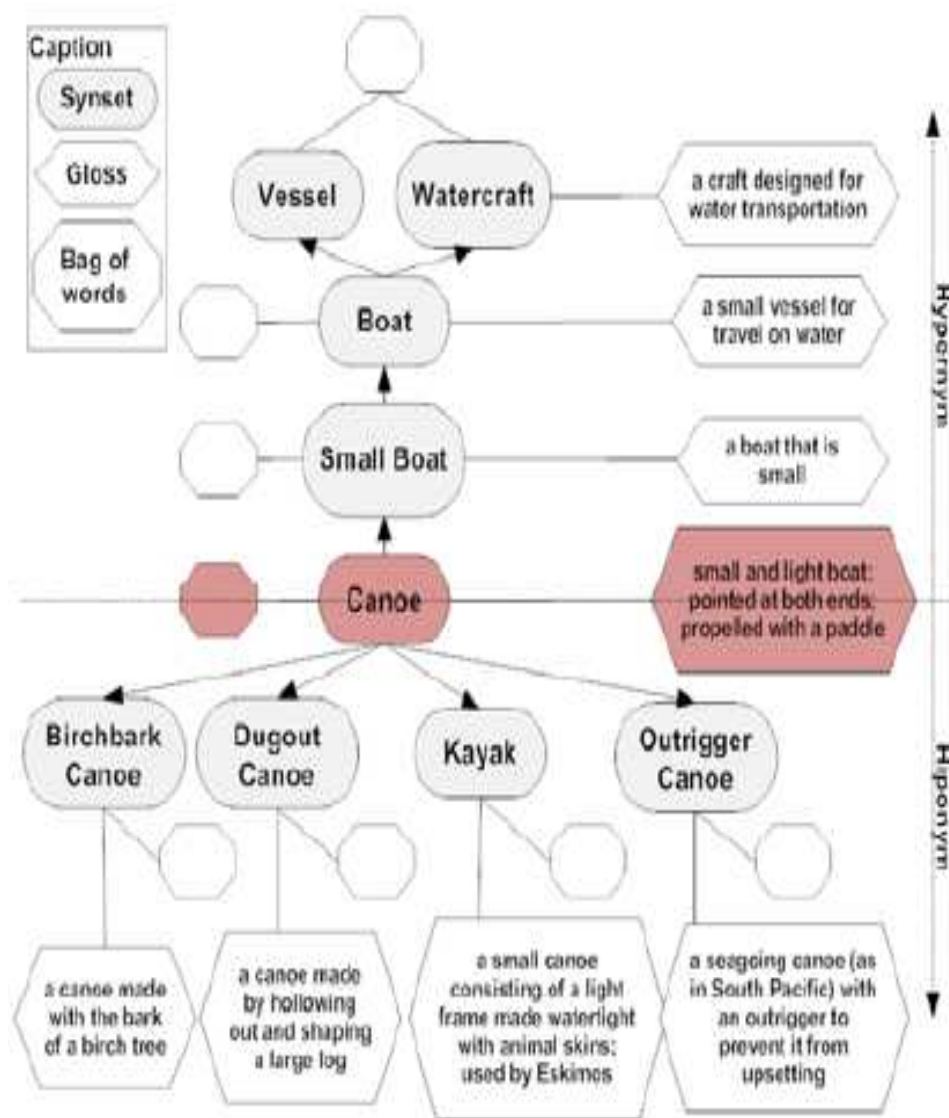


Ilustración 5 - Desglose de información

2.2. Graph Generation

Tarea *Dynamic Semantic Clustering*

- *Se encarga de que a medida que se va analizando el texto de los casos de uso, los verbos y objetos directos descubiertos se vayan agrupando dinámicamente.*
- *Para lograrlo se utiliza el **análisis de similitud** entre la palabra y las palabras de cada grupo.*
- *Para aquellos grupos donde la inclusión de esa palabra mantenga (en promedio) el puntaje de similitud en un número que supere un umbral preestablecido, se la almacena como candidata a integrarse a ese grupo.*
- *Para aquel grupo donde el puntaje es el máximo, se intenta agregarlo a este grupo.*
- *Puede darse el caso de que al agregar la palabra, el grupo deba ser dividido para que el agrupamiento sea más efectivo (cuando el promedio mínimo no se respeta, o cuando el grupo es muy disperso).*
- *También puede ocurrir que no se integre a ningún grupo en particular (cuando no existen muchos grupos o cuando la palabra no está relacionada con ningún agrupamiento) y se agregue a un grupo de un sólo elemento.*

2.2. Graph Generation

Tarea Graph Generator

- Lleva a cabo la creación del grafo propuesto anteriormente.
- Para realizarlo, se cuenta con la información generada por las tareas anteriores, principalmente con las listas y agrupamientos de verbos y objetos directos que son agregados al grafo e interconectados, de manera tal que el mismo pueda ser recorrido.
- Además, se agregan al grafo los nodos Grupo de verbo y Grupo de objetos directo (GV,GOD) que representan la acción de uno de los verbos del grupo sobre uno de los objetos directo del grupo, y los nodos de “uso” que relacionan estos nodos con un caso de uso en particular.
- Luego de esta agregación, se insertan al grafo aquellos nodos simples (verbos sin objetos directos) detectados.
- Esto representa una diferencia respecto del grafo AOIG original, y se realiza agregando los verbos simples a todos aquellos nodos (GV,GOD) cuyo grupo de verbo contenga al verbo en cuestión. Si no existe ningún nodo (GV,GOD) que lo contenga, se crea uno nuevo que no contiene ningún GOD.

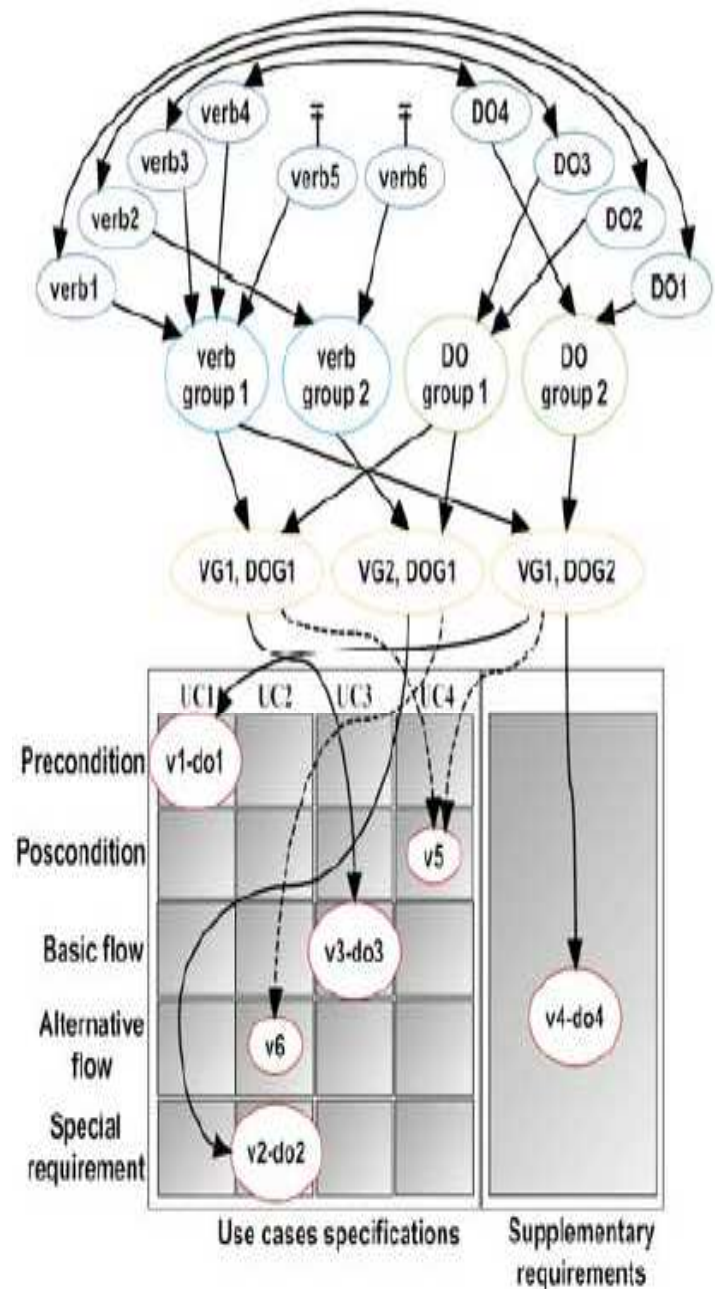
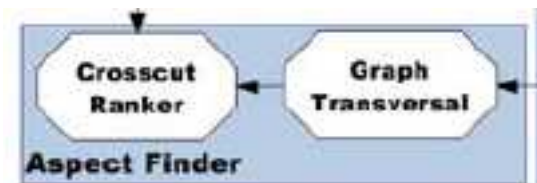


Ilustración 1 - Grafo de identificación de aspectos

2.3. Aspect Finder

- Tiene como propósito, apoyándose en los artefactos generados por las tareas previas (principalmente en el grafo), la identificación de aspectos candidatos y el ordenamiento de éstos según su relevancia.
- El bloque está formado por dos tareas que ejecutan esta elaboración:
 - *Recorrido del grafo en búsqueda de concerns crosscutting (Graph Transversal)*
 - *Ordenamiento de los aspectos candidatos (Crosscut Ranker), además de la tarea adicional filtrado de aspectos candidatos (Candidate Aspects Filtering).*



2.3. Aspect Finder

Tarea *Graph Transversal*

- Tiene la responsabilidad de **identificar los aspectos candidatos**.
- Para efectuarla, se realiza un recorrido del grafo contando aquellos grupos de verbos que cortan transversalmente una cantidad de casos de uso que supera un umbral predeterminado.
- Para cada grupo de verbo que es considerado crosscutting, se realiza una exploración en busca del término (un verbo) más representativo del grupo, utilizando las estadísticas recolectadas previamente, para proponerlo como nombre candidato del concern detectado.
- Adicionalmente, también se establece si el concern es funcional o no funcional, analizando de donde provienen los verbos.

2.3. Aspect Finder

Tarea Crosscut Ranker

- *Objetivo: establecer un ordenamiento en los grupos de verbos crosscutting detectados.*
- *Para establecer el orden de los aspectos candidatos, se toman en consideración varios parámetros de cada aspecto potencial:*
 - *el número de casos de uso cortados transversalmente*
 - *el número de cortes transversales totales (puede haber más de una referencia en un caso de uso específico) del grupo de verbos*
 - *la relevancia de ocurrencia del grupo de verbos,*
 - *la relevancia de ocurrencia de los grupos de objetos directos relacionados con el grupo de verbos en cuestión,*
 - *si el grupo de verbos es considerado funcional o no funcional.*
- *Estos parámetros son reunidos en una sola función, que fue definida de acuerdo a nuestras observaciones, cuyo resultado es un número real representando la importancia del aspecto candidato.*

El puntaje es:

$$\Delta * \left(\frac{ucc}{\sum UC} * \alpha + \frac{vgcc}{\sum UC} * \beta + \left(\frac{vgr}{\sum V} + \frac{dogsr}{\sum DO} \right) * \gamma \right)$$

Sujeto a:

$$\alpha = 10$$

$\Delta = 3$ si el grupo de verbos es no funcional

$$\beta = 6$$

$\Delta = 2$ si el grupo de verbos es funcional

$$\gamma = 3$$

Donde:

ucc : use cases crosscutted

vgcc: verb group crosscutted count

vgr: verb group relevance (occurrence count)

dogsr: direct objects groups relevance (occurrence count)

Ilustración 6 - Fórmula de ordenamiento

2.3. Aspect Finder

Tarea *Candidate Aspects Filtering*

- *Manejar la selección, especificación y administración de los aspectos candidatos por parte desarrollador, luego de que la herramienta los haya encontrado.*
- *Poder reducir la información generada por el enfoque en un subconjunto manejable basándose por ejemplo en determinados tipos de criterios.*
- Este proceso permite que se aplique en grandes desarrollos tornando la herramienta en un enfoque escalable, es decir que, sin importar la cantidad de aspectos candidatos que haya, por más grande que sea el sistema a desarrollar, el analista podrá especificarlos y manejarlos de forma eficiente y práctica.
- Para lograrlo, se utiliza la estructura del grafo generado para poder reducir la información basándose varios criterios.
 - *Verbo, en el cual se especifica el nombre de un verbo, y cada aspecto candidato que contenga este verbo será retornado.*
 - *Objeto directo, donde se especifica el nombre de un objeto directo, y cada aspecto candidato que contenga este objeto directo en alguno de sus usos será retornado.*
 - *Cuenta crosscutting, en el que se especifica un número mínimo de cuenta crosscutting, y cada aspecto candidato cuya cuenta crosscutting iguale o supere este número será retornado.*
 - *Caso de uso, donde se especifica un caso de uso y todos aquellos aspectos candidatos que contengan algún uso en este caso de uso será retornado).*

Bibliografía

- Jacobson I. and Ng P. W. Aspect-oriented Software Development with Use Cases. Addison Wesley Professional, 2005.